LipScribe presenter notes

Technical implementation presentation | CVDL Group 4

These notes go with the slide deck *LipScribe_Implementation_Presentation.pptx*. For every slide they give a suggested time, what to say, the background you need to explain it without reading, and questions you may be asked. The deck is built on the UCU template in Trebuchet MS, with 22 pt body text. A short version of each script is also in the speaker notes of the slide itself.

Read the whole document, not only your own part. Any member may be questioned on any part of the project.

Who presents what

Part	Topic	Presenter	Slides	Time
1	Overview, architecture and tools	Cole Rodney Migisha	1–12	9:55
2	Environment and data	Charis Opol	13–21	8:25
3	Visual and audio pipelines	Ainomugisha Faith	22–29	7:40
4	The multimodal model	Lisa Lusinga Namugenyi	30–38	8:40
5	Training and evaluation	Ansasire Nehum Naamaye	39–47	8:55
6	Application, plan and risks	Wakisa Isabelle	48–61	10:05
	Total		1–61	53:40

Part 1 also opens the talk (title and agenda). Part 6 also closes it (references, thank you) and chairs questions.

Timing

No time limit was given for this presentation, so the suggested times assume about one minute per content slide and ten seconds per divider, which gives roughly eight to ten minutes per presenter. If your slot is shorter, scale every time down by the same factor and spend the saved time on the slides marked with the longest times. Those are the architecture, data split, fusion, CTC and evaluation slides, which carry the most technical weight.

The one-sentence version

Captioning tools that only listen fail in noise. LipScribe also reads the lips, and learns through cross-modal attention and modality dropout to lean on them when the audio degrades. We will measure this on held-out GRID speakers at five noise levels against three baselines.

Agree these as a group before you present

- **There are no results yet.** This is the implementation plan. The results table is deliberately empty. Never quote a WER as achieved.
- **Targets are hypotheses:** WER below 10% on clean GRID audio, and a clear advantage over audio-only at 0 and -5 dB.
- **Data:** GRID, 34 speakers x 1,000 sentences, about 33,000 usable, 51-word vocabulary, 25 fps, about 75 frames per 3 s clip.

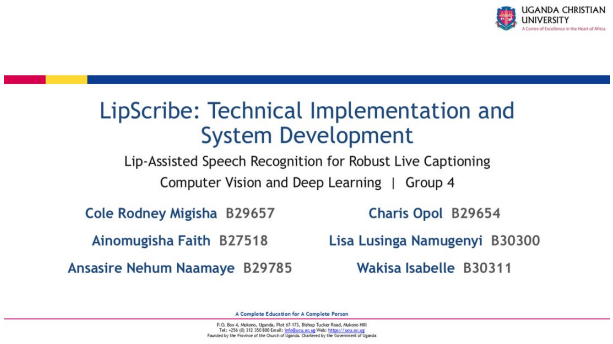
- **Noise:** MUSAN noise subset at clean, 10, 5, 0 and -5 dB SNR, with disjoint noise-file pools for train, validation and test.
- **Split:** by speaker. About 27 train, 3 validation, 4 test (2 female, 2 male). Never by sentence.
- **Shapes:** mouth crops 96 x 96 grey; both encoders output B x 75 x 512; fusion width 256; 28 output symbols (blank, space, a-z).
- **Labels in the document:** [Proposal] = fixed by the proposal; [Recommended] = our engineering choice, may change; [To be decided] = settled by experiment.
- **Handover line:** each presenter ends with one sentence naming the next presenter and their topic. These lines are already in the scripts.

Part 1: Overview, architecture and tools

Presenter: **Cole Rodney Migisha**

Slides 1–12 | suggested total 9:55

Slide 1 | LipScribe: Technical Implementation (0:30)



WHAT TO SAY

- Good afternoon. We are Group 4 and this is LipScribe: lip-assisted speech recognition for robust live captioning.
- Last time we presented the proposal. Today we present the technical implementation plan: how the system will actually be built, trained, tested and demonstrated.
- Each of us presents one part, so you will hear from all six of us.

BACKGROUND YOU SHOULD UNDERSTAND

This talk follows the Technical Implementation and System Development Document. It describes planned work, not completed work: there are no experimental results yet, and every number we have not measured is a recommendation or target.

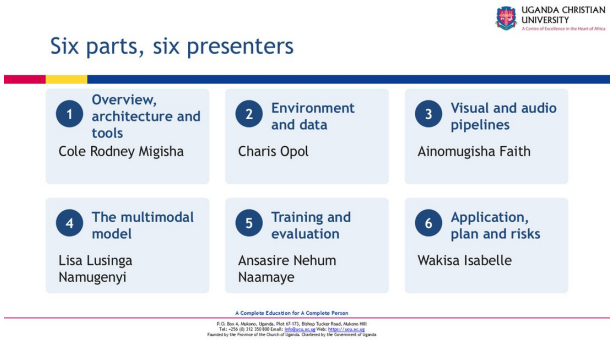
Use the three statement labels from the document if you are challenged on a number: [Proposal] means fixed by our proposal, [Recommended] means an engineering choice we propose and may revise, [To be decided] means an open question we will settle by experiment.

QUESTIONS YOU MAY BE ASKED

Do you have any results yet?

No. This is the implementation plan. The results table on slide 45 is deliberately empty and will be filled from logged runs.

Slide 2 | Six parts, six presenters (0:30)



WHAT TO SAY

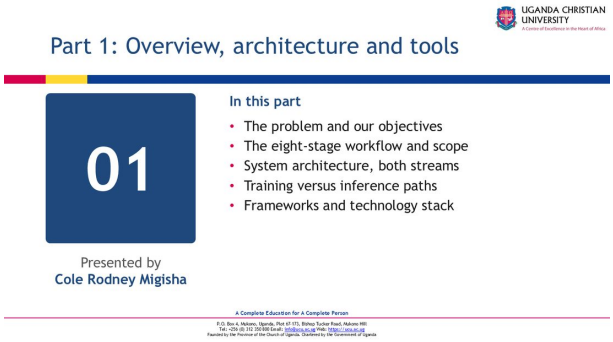
- The talk has six parts.
- I cover the overview, architecture and tools. Charis covers the environment and the data. Faith covers the visual and audio pipelines. Lisa covers the multimodal model. Nehum covers training and evaluation. Isabelle covers the application, the schedule, the risks, and closes.
- Each part follows the order of the implementation document.

BACKGROUND YOU SHOULD UNDERSTAND

Handovers: each part starts with a numbered divider slide naming the presenter. The outgoing speaker says one sentence handing over, for example: 'Charis will now explain the data we train on.'

The requirements say any member may be questioned on any part of the project, so every presenter should read all six parts of these notes, not only their own.

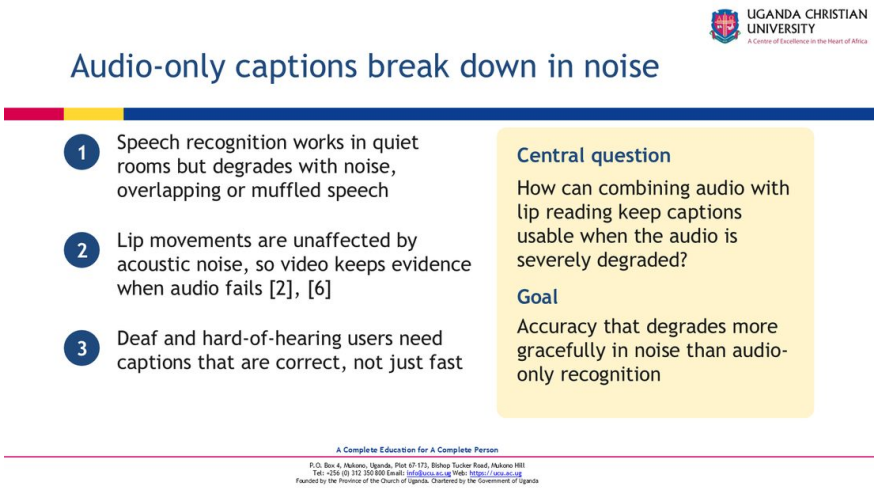
Slide 3 | Part 1: Overview, architecture and tools (0:10)



WHAT TO SAY

- Part 1: the problem, the objectives, the overall architecture and the tools we will use.

Slide 4 | Audio-only captions break down in noise (1:00)



WHAT TO SAY

- Ordinary speech recognition works well in a quiet room but degrades sharply with background noise, overlapping speakers or muffled speech.
- Lip movements are not affected by acoustic noise. A model that also watches the mouth keeps a usable source of evidence when the audio becomes unreliable.
- Our users are deaf and hard-of-hearing people in lectures, meetings and public spaces. They need captions that are correct, not just fast.
- So our central question is: how can combining audio with lip reading keep captions usable when the audio is severely degraded? Our goal is accuracy that degrades more gracefully in noise than audio-only recognition.

BACKGROUND YOU SHOULD UNDERSTAND

Automatic speech recognition (ASR) turns audio into text. Audio-visual speech recognition (AVSR) uses both the audio and a video of the speaker's mouth. LipScribe is an AVSR system.

Why correctness over speed: a caption that arrives quickly but is wrong misleads the user. That is why the plan treats accuracy across noise levels as the main measure and treats latency as a constraint to measure, not to assume.

References: [2] Li et al. 2023 show that improving lip reading improves recognition in noise; [6] Afouras et al. 2022 is the standard deep AVSR reference.

QUESTIONS YOU MAY BE ASKED

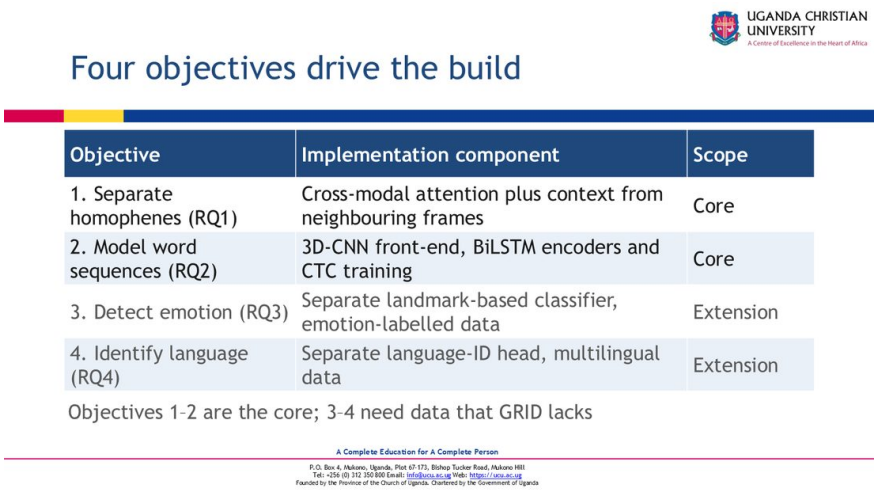
Why is this a computer vision project and not a speech project?

The information that survives noise is visual. The new work is locating the mouth in video, encoding its shape and motion over time, and fusing that with audio. The audio-only system is our baseline to beat.

What does 'degrade more gracefully' mean in measurable terms?

Word error rate should rise more slowly as the signal-to-noise ratio falls. We measure this at five noise levels, and the gap to the audio-only baseline should grow at 0 and -5 dB.

Slide 5 | Four objectives drive the build (1:00)



WHAT TO SAY

- The proposal set four objectives, each tied to a research question.
- Objective 1 is to separate homophenes, words that look the same on the lips. Cross-modal attention lets the audio resolve them, helped by context from neighbouring frames.
- Objective 2 is to model whole word sequences. The 3D-CNN front-end, the BiLSTM encoders and CTC training map frame sequences to sentences.
- Objectives 3 and 4, emotion detection and language identification, are extensions. GRID has no emotion labels and only one language, so they are separate modules with their own data.

BACKGROUND YOU SHOULD UNDERSTAND

Homophenes are sounds that look identical on the lips, such as /p/, /b/ and /m/. Lips alone cannot tell them apart, but the audio usually can. Equally, sounds that are confused in noise often look different on the lips. Cross-attention lets each stream borrow evidence from the other.

RQ = research question. RQ1 homophenes, RQ2 temporal sequences, RQ3 emotion, RQ4 language.

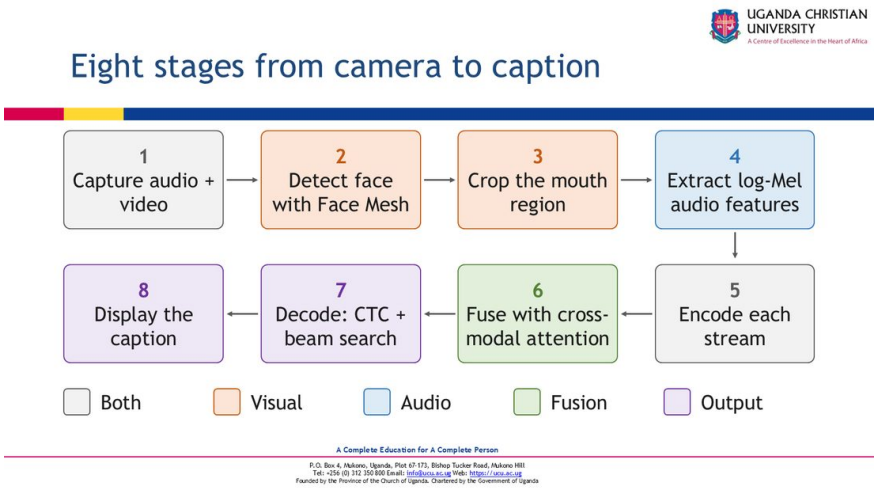
Keeping objectives 3 and 4 separate means an unvalidated extension can never delay or obscure the main result.

QUESTIONS YOU MAY BE ASKED

Why not drop objectives 3 and 4 entirely?

They were in the proposal, so we keep a plan for them, but honestly scoped: separate modules, separate data, attempted only after the core freeze at the end of week 5.

Slide 6 | Eight stages from camera to caption (1:00)



WHAT TO SAY

- At run time every segment of input goes through the same eight stages.
- One: capture synchronised video and audio. Two: detect the face with MediaPipe Face Mesh. Three: crop a fixed-size region around the lips. Four: compute log-Mel audio features.
- Five: encode each stream with its own network. Six: fuse the two with cross-modal attention. Seven: decode the fused sequence into characters with CTC and beam search. Eight: display the caption.
- The colours used here are the same in every diagram: orange for visual, blue for audio, green for fusion and purple for output.

BACKGROUND YOU SHOULD UNDERSTAND

MediaPipe Face Mesh [14], [15] is a pretrained model that returns 468 3D landmarks on the face for each frame. We only use the lip landmarks to place the crop.

Log-Mel features describe how much energy the audio has in each frequency band over time, on a scale that matches human hearing. Part 3 explains them.

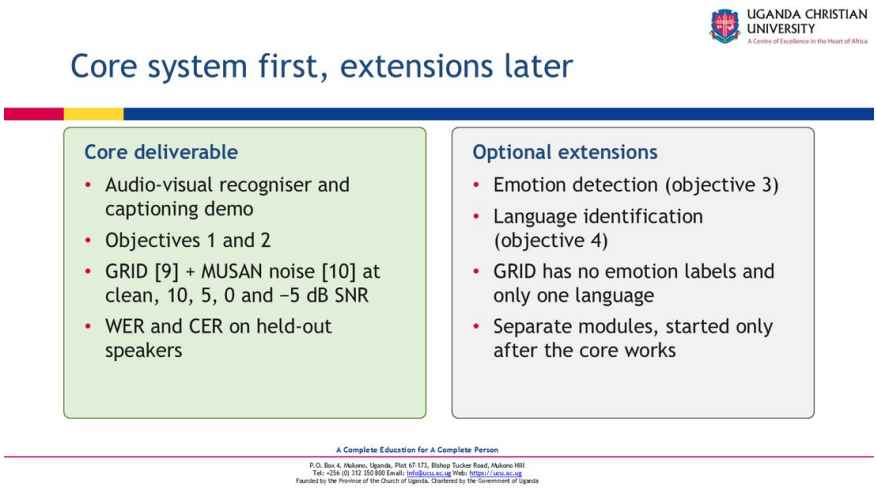
CTC (Connectionist Temporal Classification) lets the network output text without being told which frame each character belongs to. Part 4 explains it.

QUESTIONS YOU MAY BE ASKED

Which stages are learned and which are fixed?

Stages 2 and 3 use a pretrained, frozen Face Mesh plus geometry. Stage 4 is a fixed signal transform. Stages 5 to 7 are the network we train. Beam search in stage 7 is an algorithm, not learned.

Slide 7 | Core system first, extensions later (0:45)



WHAT TO SAY

- The core deliverable is the audio-visual recogniser and a captioning demo built on it.
- It covers objectives 1 and 2 and the evaluation in the proposal: training on GRID with MUSAN noise added at clean, 10, 5, 0 and -5 dB, and reporting WER and CER on held-out speakers.
- Emotion detection and language identification are optional extensions, started only after the core works.

BACKGROUND YOU SHOULD UNDERSTAND

SNR (signal-to-noise ratio) in decibels compares speech power with noise power. 10 dB means speech is 10 times more powerful than noise; 0 dB means they are equal; -5 dB means the noise is about three times stronger than the speech.

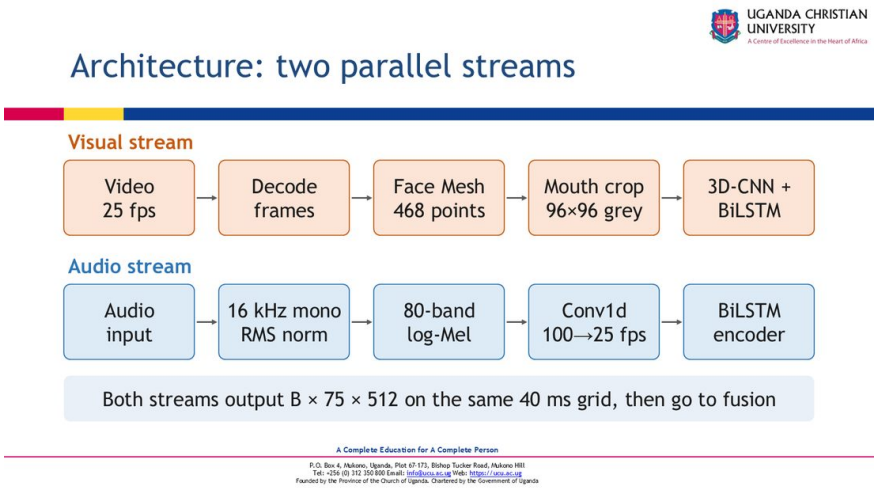
WER and CER (word and character error rate) are defined in Part 5. 'Held-out speakers' means people the model never saw during training.

QUESTIONS YOU MAY BE ASKED

Why GRID and not a larger, more realistic dataset?

GRID is clean, aligned, frontal and small enough to train in six weeks. Its fixed grammar limits how far results generalise, and we say so in the risks.

Slide 8 | Architecture: two parallel streams (1:15)



WHAT TO SAY

- LipScribe is a pipeline of independent modules with fixed tensor interfaces.
- The visual stream reads video at 25 frames per second, decodes the frames, finds 468 face landmarks, crops a 96 by 96 grey-scale mouth image, and encodes the sequence with a 3D-CNN and a BiLSTM.
- The audio stream takes the audio, converts it to 16 kHz mono at a fixed loudness, computes 80-band log-Mel features, subsamples them from 100 to 25 frames per second with strided 1D convolutions, and encodes them with a BiLSTM.
- Both streams end with the same shape, batch by 75 frames by 512 features, on the same 40 millisecond grid. That makes fusion simple.

BACKGROUND YOU SHOULD UNDERSTAND

B is the batch size, 75 is the number of frames in a 3 second clip at 25 fps, 512 is the feature size (256 per LSTM direction, forward and backward concatenated).

Why modules: each one can be developed, tested and replaced on its own, team members can work in parallel, and the baselines reuse the same components so each comparison isolates one change.

Audio and video are captured together; sample n of the audio belongs to video frame floor(25n/16000). This is how the streams stay in sync.

QUESTIONS YOU MAY BE ASKED

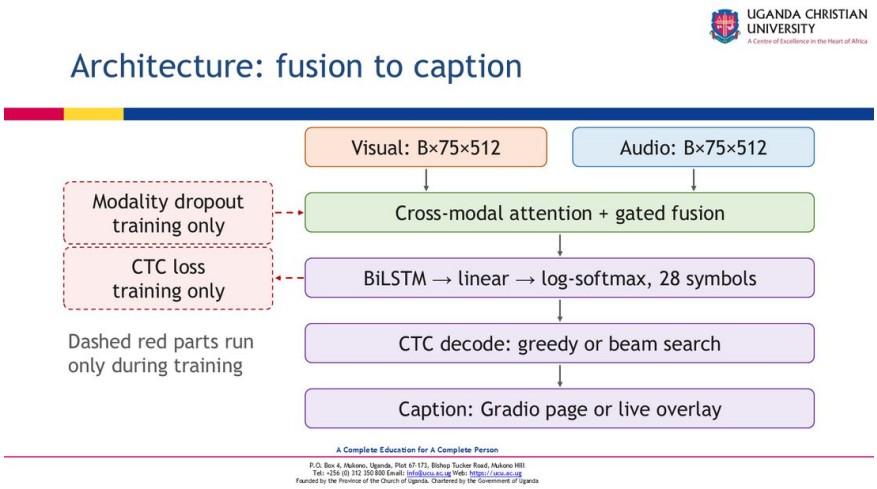
Why are both streams brought to exactly 75 frames?

So frame t of the audio encoding and frame t of the video encoding describe the same 40 ms instant. The per-frame fusion gate needs equal lengths.

What is a BiLSTM?

A recurrent network that reads the sequence forwards and backwards and concatenates both, so each frame's output uses past and future context.

Slide 9 | Architecture: fusion to caption (1:00)



WHAT TO SAY

- The two 75 by 512 sequences enter the fusion module: cross-modal attention followed by a learned gate.
- The fused sequence goes through a small BiLSTM, a linear layer and a log-softmax over 28 symbols: 26 letters, space and the CTC blank.
- Decoding turns those per-frame probabilities into text, greedily during validation and with beam search for final results and the demo. The caption is shown in a Gradio page or a live overlay.
- The dashed red parts exist only during training: modality dropout, which sometimes hides one stream, and the CTC loss.

BACKGROUND YOU SHOULD UNDERSTAND

Log-softmax gives log-probabilities for each of the 28 symbols at each frame. CTC loss is computed from these during training; at inference they are decoded instead.

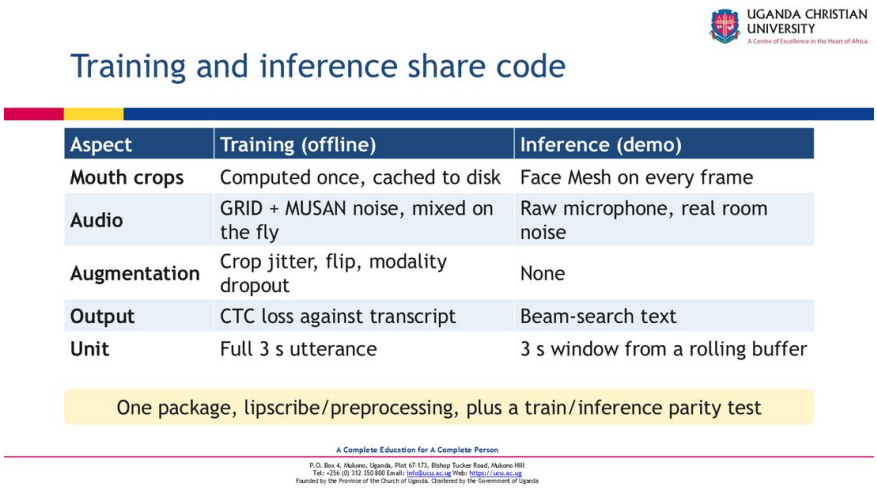
Modality dropout is explained by Lisa in Part 4. In short: randomly removing the audio or the video during training forces the model to use both.

QUESTIONS YOU MAY BE ASKED

Why have a BiLSTM after fusion as well?

Fusion is per frame. The extra BiLSTM lets the fused representation use context across the whole utterance before the output layer.

Slide 10 | Training and inference share code (1:00)



WHAT TO SAY

- The same model code runs in training and at inference, but the data path around it differs.
- In training, mouth crops are computed once and cached; at inference Face Mesh runs on every frame. In training, clean GRID audio is mixed with MUSAN noise on the fly; at inference the noise is whatever the room contains.
- Augmentation only happens in training. Training computes the CTC loss; inference produces beam-search text. Training uses whole 3 second utterances; the demo uses 3 second windows from a rolling buffer.
- All shared preprocessing lives in one package, lipscribe/preprocessing, and a parity test checks the live path gives the same tensors as the cached training features.

BACKGROUND YOU SHOULD UNDERSTAND

The risk this prevents: preprocessing at demo time silently differs from training (different crop, different normalisation), so the demo performs worse than the evaluated model for reasons that have nothing to do with the model.

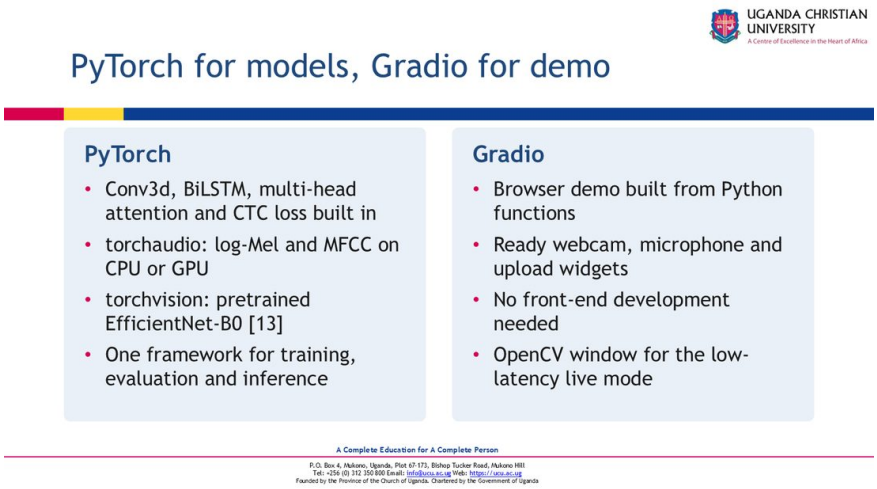
Also different: training runs model.train() (dropout and batch-norm updates active); inference runs model.eval() inside torch.inference_mode(). Normalisation statistics are computed on the training split and loaded from file at inference.

QUESTIONS YOU MAY BE ASKED

What exactly does the parity test compare?

A GRID clip is processed through the live preprocessing path and compared with its cached training features. The maximum difference must be below 1e-5.

Slide 11 | PyTorch for models, Gradio for demo (1:00)



WHAT TO SAY

- We use Python with one tool per role.
- PyTorch has every component we need built in: 3D convolutions, bidirectional LSTMs, multi-head attention and CTC loss. torchaudio computes log-Mel and MFCC features identically on CPU and GPU, and torchvision supplies the pretrained EfficientNet-B0 for one experiment. One framework covers training, evaluation and inference.
- For the interface we use Gradio. It builds a browser demo straight from Python functions, with ready webcam, microphone and upload components, so no front-end development is needed.
- For the lower-latency live mode we use an OpenCV display window driven by the same inference code.

BACKGROUND YOU SHOULD UNDERSTAND

PyTorch's eager execution makes tensor shapes easy to inspect while debugging. TensorFlow would also work; we chose PyTorch so that one framework covers everything.

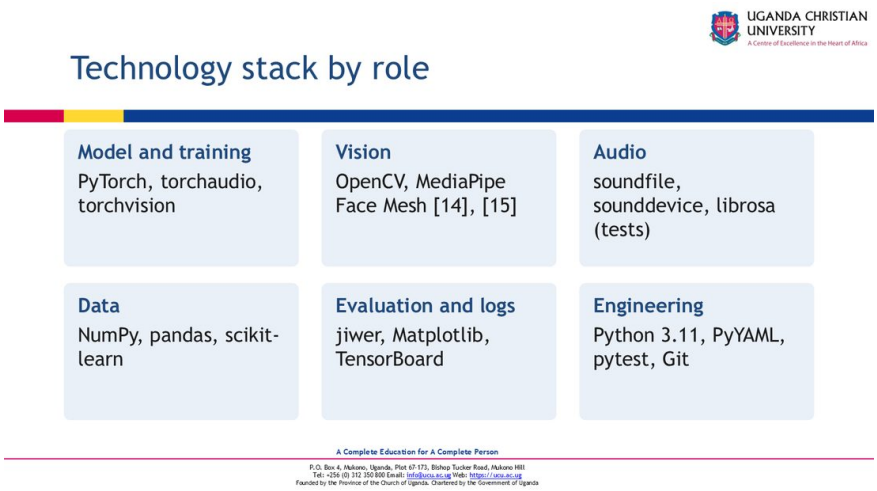
The OpenCV live window is a development tool rather than a second interface framework: it avoids browser round-trips that add latency.

QUESTIONS YOU MAY BE ASKED

Why not build a mobile app?

The proposal promised a simple web demonstration. Gradio meets that in days, not weeks, and keeps our effort on the model.

Slide 12 | Technology stack by role (0:45)



WHAT TO SAY

- This is the full stack grouped by role.
- Model and training: PyTorch, torchaudio and torchvision. Vision: OpenCV for video and display, MediaPipe Face Mesh for landmarks. Audio: soundfile for WAV files, sounddevice for the live microphone, librosa only to cross-check our features in tests.
- Data: NumPy, pandas for the dataset manifest, scikit-learn for speaker-grouped splitting. Evaluation and logs: jiwer for error rates, Matplotlib for plots, TensorBoard for live training curves. Engineering: Python 3.11, PyYAML for configs, pytest for tests, and Git with GitHub.
- Charis will now explain the hardware and the data.

BACKGROUND YOU SHOULD UNDERSTAND

Gradio is also part of the stack (previous slide). scikit-learn's GroupShuffleSplit splits by speaker; it is also used for confusion matrices in the extensions.

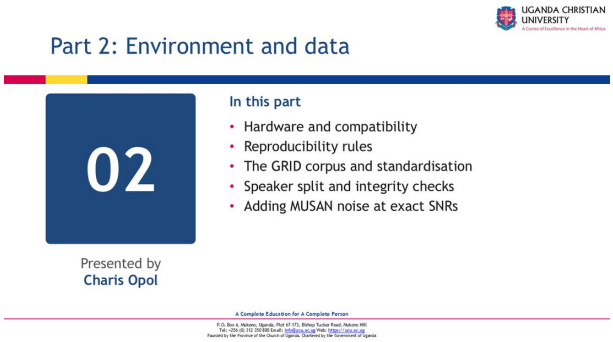
jiwer computes corpus-level WER and CER, the convention we use for reporting.

Part 2: Environment and data

Presenter: **Charis Opol**

Slides 13–21 | suggested total 8:25

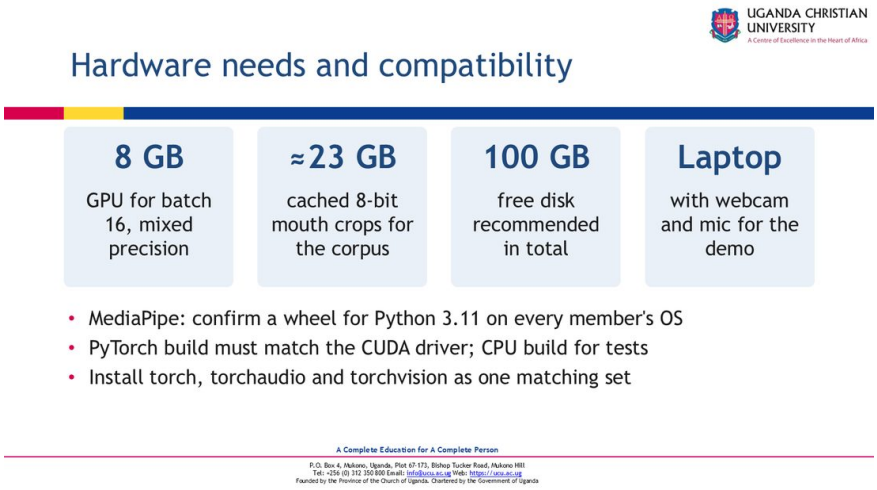
Slide 13 | Part 2: Environment and data (0:10)



WHAT TO SAY

- Part 2: the environment we need and the data we train on.

Slide 14 | Hardware needs and compatibility (1:00)



WHAT TO SAY

- We have not yet confirmed which hardware is available, so the plan states requirements rather than assuming a machine.
- Training needs a CUDA GPU; on a CPU alone it would not finish in six weeks. With about 8 GB of GPU memory, a batch of 16 clips with mixed precision should fit. We confirm that with a memory test in week 2.
- Cached mouth crops for the whole corpus take about 23 GB, and with the raw corpus and MUSAN we recommend about 100 GB of free disk. The demo only needs a laptop with a webcam and microphone.
- Three compatibility checks: MediaPipe must have a wheel for Python 3.11 on every member's operating system; the PyTorch build must match the GPU's CUDA driver; and torch, torchaudio and torchvision must be installed as a matching set.

BACKGROUND YOU SHOULD UNDERSTAND

Where 23 GB comes from: about 33,000 clips x 75 frames x 96 x 96 pixels x 1 byte (8-bit grey) is about 22.8 GB.

Mixed precision (AMP) runs most operations in 16-bit floating point, roughly halving memory use and speeding up training on modern GPUs.

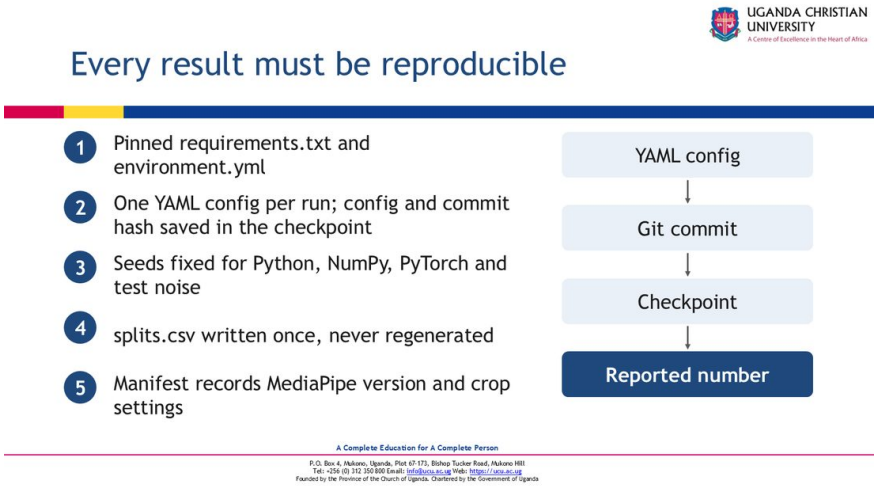
GPU source is still to be decided: a university workstation, a team member's GPU, or a cloud notebook service.

QUESTIONS YOU MAY BE ASKED

What if you get no GPU?

We develop on a subset of speakers, keep the model moderate, and drop optional experiments first. CPU is enough for preprocessing, tests and the demo, but not for full training.

Slide 15 | Every result must be reproducible (1:00)



WHAT TO SAY

- Every number we report must be traceable.
- Exact dependency versions are pinned in requirements.txt and environment.yml, generated from a working environment. Every experiment is one YAML configuration file committed to Git, and the configuration and commit hash are saved inside each checkpoint.
- Seeds are fixed for Python, NumPy and PyTorch, and the noise-mixing seed for the test set is fixed, so every model sees identical noisy audio. The speaker split is written once to splits.csv and never regenerated. The manifest records the MediaPipe version and crop settings.
- The chain on the right is the rule: a reported number must trace back to a checkpoint, a commit and a config.

BACKGROUND YOU SHOULD UNDERSTAND

An experiment is reported only if its configuration file is committed. Run directories are never overwritten.

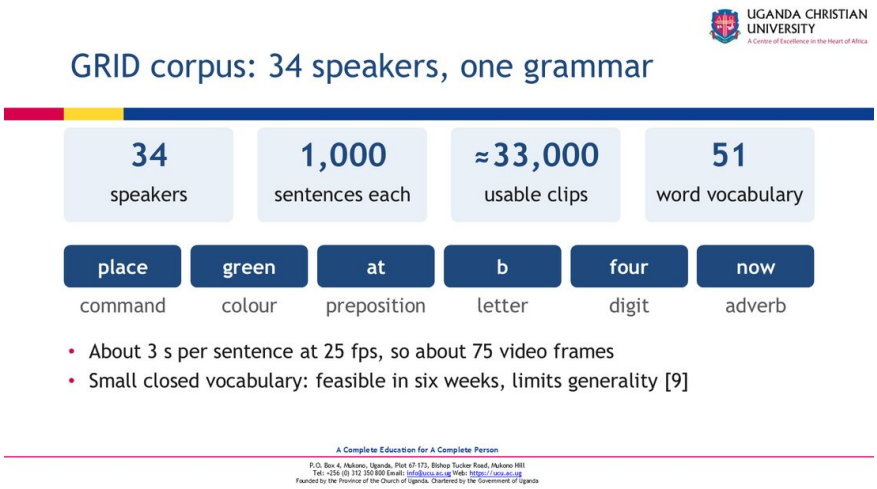
Why fix the test noise seed: if each model were tested on differently mixed audio, part of the difference between models would be noise luck rather than the models.

QUESTIONS YOU MAY BE ASKED

How would someone outside the team reproduce a result?

Clone the repository, install from requirements.txt, run the data scripts, then run evaluate.py on the released checkpoint. A team member not involved in setup will do exactly this as a deliverable check.

Slide 16 | GRID corpus: 34 speakers, one grammar (1:00)



WHAT TO SAY

- Our primary dataset is the GRID corpus [9]: audio and frontal-face video of 34 speakers, each saying 1,000 short sentences. About 33,000 are usable.
- Every sentence follows the same six-slot grammar: command, colour, preposition, letter, digit, adverb. For example, 'place green at b four now'.
- A sentence lasts about 3 seconds, which at 25 fps is about 75 video frames, and each comes with a word-level alignment file from which we build the transcript.
- The vocabulary is small and closed: 51 words. That makes the task feasible in six weeks, but it also limits how far our conclusions generalise to open conversation.

BACKGROUND YOU SHOULD UNDERSTAND

The alignment files give start and end times for each word; we only use them to build transcripts (silence and short-pause tokens removed).

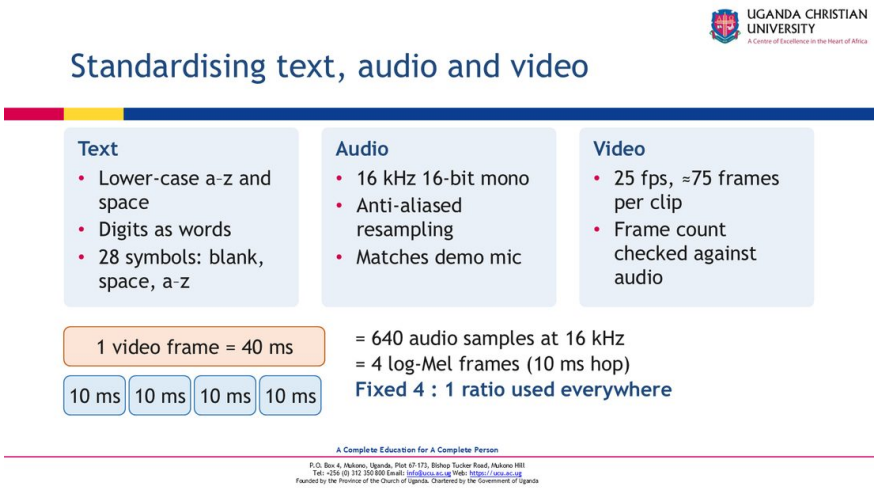
The integrity checks on slide 19 decide how many of the ~33,000 sentences we actually keep. We will quote those counts, not assume them.

QUESTIONS YOU MAY BE ASKED

Isn't GRID too easy?

Yes, compared with real captioning, and we say so. Its value is a controlled setting in which the effect of noise and of adding video can be measured cleanly. Larger datasets are future work.

Slide 17 | Standardising text, audio and video (1:00)



WHAT TO SAY

- Before training we standardise all three kinds of data.
- Text: lower-case letters a to z and space only. The letter slot stays one character, like 'b', and digits are words, like 'four', matching the alignment files. That gives 28 output symbols: the CTC blank at index 0, space, and 26 letters.
- Audio: 16 kHz, 16-bit mono WAV, resampled with an anti-aliasing filter. 16 kHz is the standard rate for speech and is what the demo microphone delivers.
- Video: 25 fps, so about 75 frames per clip, with the frame count checked against the audio. The key relation: one video frame is 40 ms, which is 640 audio samples and exactly 4 log-Mel frames. We use that fixed 4 to 1 ratio everywhere.

BACKGROUND YOU SHOULD UNDERSTAND

Why characters, not words: character output keeps CER meaningful, lets the architecture be reused for other text later, and matches the CTC systems in the proposal [4], [6].

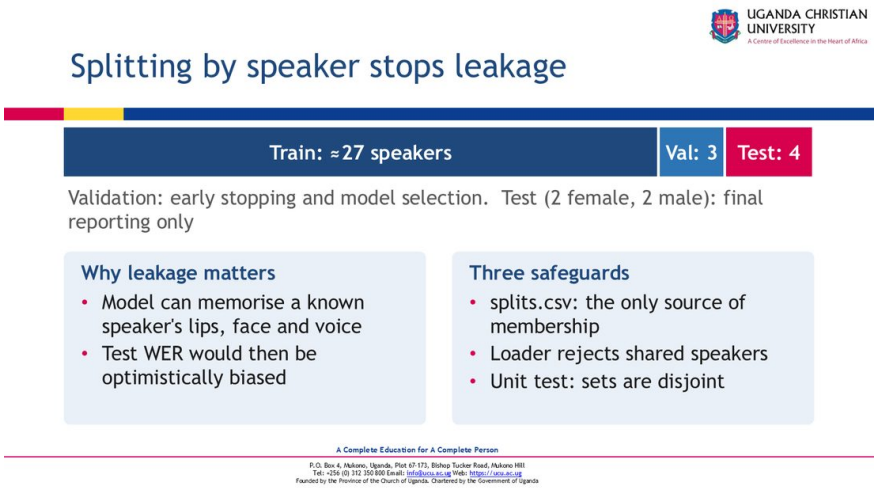
40 ms x 16,000 samples/s = 640 samples. Audio features use a 10 ms hop, so 4 feature frames per video frame.

QUESTIONS YOU MAY BE ASKED

Why not predict whole words, since GRID has only 51?

A word classifier would only ever work on GRID. Characters keep the model general and keep CER meaningful.

Slide 18 | Splitting by speaker stops leakage (1:15)



WHAT TO SAY

- The proposal requires evaluation on held-out speakers, so we split by speaker, not by sentence.
- We recommend 4 test speakers, 2 female and 2 male, used only for final reporting; 3 validation speakers for early stopping and model selection; and about 27 for training.
- Why it matters: if one person's sentences appear in both training and test, the model can partly memorise their lips, face and voice. Test error would then measure familiar people, not new users, and every WER would be optimistically biased.
- Three safeguards: splits.csv is the only source of partition membership, the data loader refuses any speaker that appears in two partitions, and a unit test asserts the three sets are disjoint.

BACKGROUND YOU SHOULD UNDERSTAND

Speakers with missing or corrupted video are excluded before the split is drawn. The split is drawn once with a fixed seed, reviewed for gender balance, and committed.

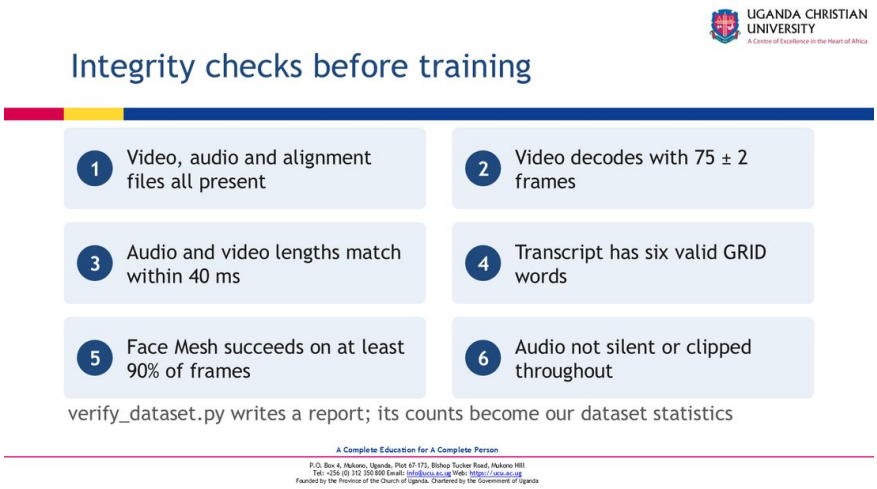
Normalisation statistics are computed on training speakers only, for the same reason: no information from test speakers may influence training.

QUESTIONS YOU MAY BE ASKED

With only 4 test speakers, how reliable are the results?

Limited, and we say so. We report per-speaker WER, use three seeds where compute allows, and use a paired bootstrap to give a confidence interval for the key comparison.

Slide 19 | Integrity checks before training (0:45)



WHAT TO SAY

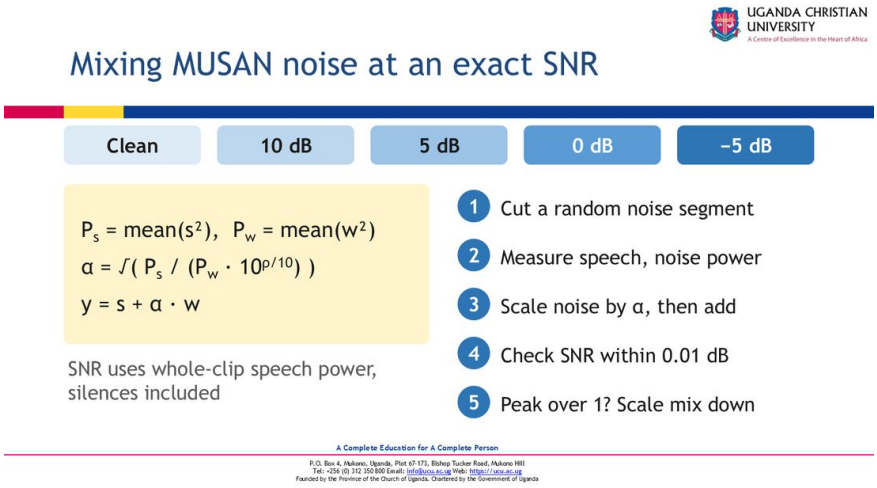
- A script, verify_dataset.py, runs before any training and checks six things.
- Every clip has video, audio and alignment files. Video decodes with 75 plus or minus 2 frames. Audio and video durations match within one frame, 40 ms. The transcript has exactly six valid GRID words. Face Mesh succeeds on at least 90% of frames. And the audio is not silent or clipped throughout.
- The report counts how many clips pass each check. Those counts become the dataset statistics in our final report.

BACKGROUND YOU SHOULD UNDERSTAND

Utterances below the 90% Face Mesh threshold are flagged and excluded. The 90% threshold is a recommendation.

Silence and clipping are detected with RMS (average level) and peak checks.

Slide 20 | Mixing MUSAN noise at an exact SNR (1:15)



WHAT TO SAY

- We add noise from the MUSAN corpus [10] at the five conditions in the proposal: clean, 10, 5, 0 and -5 dB.
- The procedure: cut a random noise segment the same length as the speech, measure the power of both, scale the noise by alpha, and add it.

- Alpha is chosen so the signal-to-noise ratio is exactly the target rho: the square root of speech power over noise power times ten to the rho over ten. The code then re-measures the SNR and asserts it is within 0.01 dB.
- If the mixture's peak exceeds 1, the whole mixture is scaled down; speech and noise scale together, so the SNR does not change.

BACKGROUND YOU SHOULD UNDERSTAND

Power is the mean of the squared samples. SNR in dB = 10 log10(Ps / Pn). Solving 10 log10(Ps / (alpha^2 Pw)) = rho for alpha gives the formula on the slide.

We use MUSAN's noise subset, which matches the 'background noise' scenario. Babble noise from the speech subset is a possible later experiment.

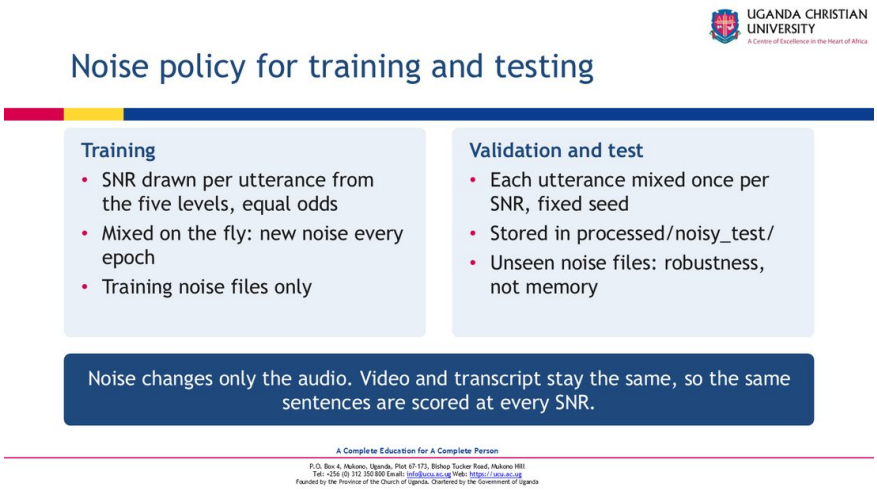
Speech power is measured over the whole clip, including the short silences at the start and end. This is simple and reproducible, but it means the SNR during active speech is somewhat higher than the nominal value. The report will state this. Measuring over voiced frames only is an option (to be decided).

QUESTIONS YOU MAY BE ASKED

If the noise file is shorter than the speech, what happens?

The noise file is looped until it is long enough, then a segment is cut at a random offset.

Slide 21 | Noise policy for training and testing (1:00)



WHAT TO SAY

- Training and testing use noise differently.
- In training, each time a clip is loaded an SNR is drawn from the five levels with equal probability and noise is mixed on the fly, so the model sees each sentence under many conditions.
- For validation and test, each clip is mixed once per SNR with a fixed seed and stored, so every model is evaluated on exactly the same audio. MUSAN noise files are split into disjoint training, validation and test pools, so testing measures robustness to unseen noise rather than memorised noise.
- Noise changes only the audio. The video and transcript stay the same, which is what lets us score the same sentences at every SNR and compare directly.
- Faith will now take you through the visual and audio pipelines.

BACKGROUND YOU SHOULD UNDERSTAND

Because noise changes every epoch, log-Mel features are computed on the fly on the GPU rather than cached. Mouth crops, which never change, are cached.

Noisy test copies are stored in data/processed/noisy_test/<snr>/.

QUESTIONS YOU MAY BE ASKED

Why not train only on clean audio and test on noise?

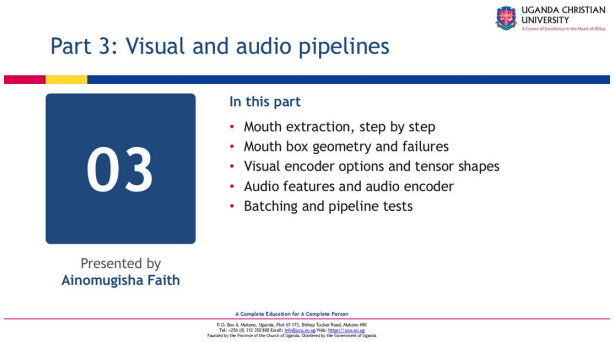
A model trained only on clean audio would be unrealistically weak in noise, and the audio-only baseline would be an unfair comparison. All models, including the audio-only baseline, train with the same multi-condition noise.

Part 3: Visual and audio pipelines

Presenter: **Ainomugisha Faith**

Slides 22–29 | suggested total 7:40

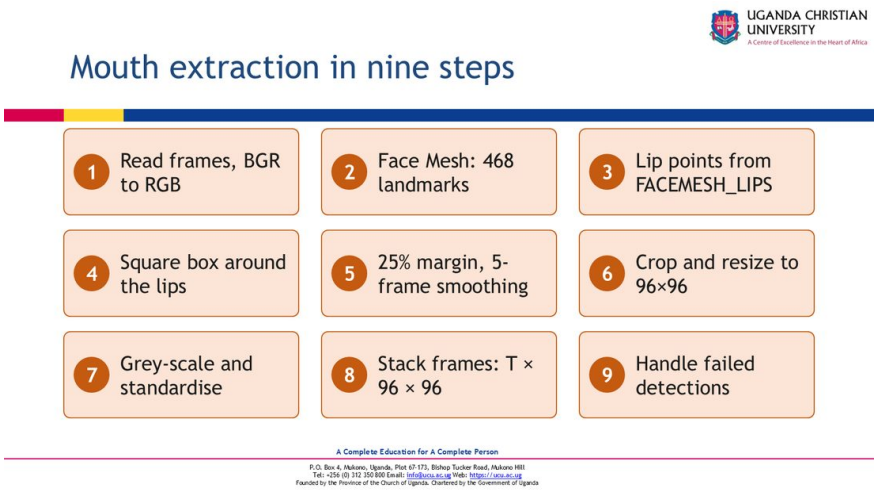
Slide 22 | Part 3: Visual and audio pipelines (0:10)



WHAT TO SAY

- Part 3: how video becomes mouth crops and features, and how audio becomes log-Mel features.

Slide 23 | Mouth extraction in nine steps (1:15)



WHAT TO SAY

- Mouth extraction runs once per clip and is cached. It has nine steps.
- One, read frames in order and convert BGR to RGB, which MediaPipe expects. Two, Face Mesh finds 468 landmarks, in video mode with one face so landmarks are tracked frame to frame. Three, take the lip landmarks from MediaPipe's own FACEMESH_LIPS list rather than typing indices by hand.
- Four, compute a square box around the lips. Five, add a 25% margin and smooth the box over 5 frames. Six, crop and resize to 96 by 96.
- Seven, convert to grey-scale and standardise. Eight, stack the crops in order into a T by 96 by 96 array. Nine, handle failed detections, which I cover on the next slide.

BACKGROUND YOU SHOULD UNDERSTAND

Tracking mode is faster and more temporally stable than detecting the face independently in each frame.

A square box is used so resizing does not distort the mouth's shape. Grey-scale removes colour differences between speakers and cameras, which carry little information about speech, and cuts

storage threefold.

Resizing uses area interpolation. Crops are stored as 8-bit integers and scaled to [0, 1] then standardised with the training-set mean and standard deviation at load time.

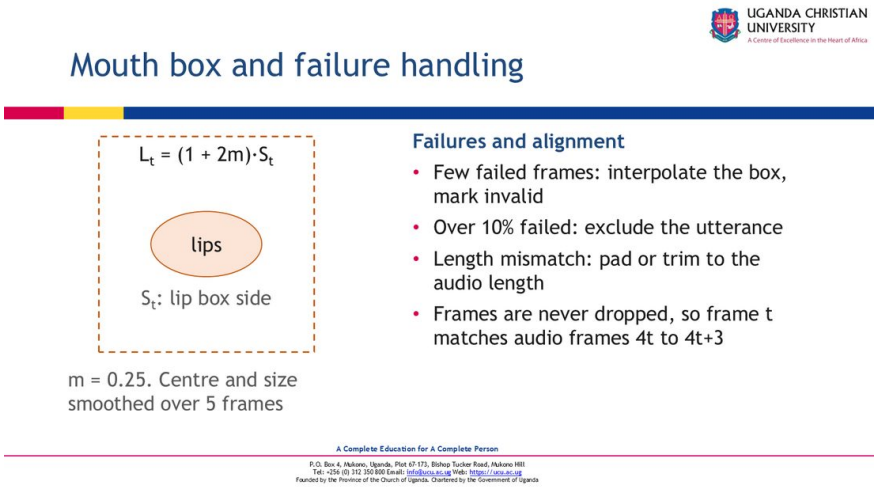
Augmentation keeps temporal order: the same random crop offset and the same horizontal flip are applied to every frame of a clip.

QUESTIONS YOU MAY BE ASKED

Why 96 by 96 and not 112 by 112?

The proposal allows either. 96 by 96 cuts input size by about 27% and is comparable to crop sizes used in sentence-level lipreading on GRID [12]. 112 remains an option if the EfficientNet experiment benefits.

Slide 24 | Mouth box and failure handling (1:00)



WHAT TO SAY

- The box is centred on the lip landmarks and its side S is the larger of the lip width and height. We enlarge it to L equals 1 plus 2m times S, with a margin m of 0.25, so the chin, mouth corners and jaw stay visible when the mouth opens.
- Raw boxes jitter by a few pixels, which the network would see as fake motion, so centre and size are smoothed with a 5-frame moving average. In the live demo we use an exponential moving average over past frames instead.
- Failures: a few failed frames get an interpolated box and are marked invalid. If more than 10% fail, the clip is excluded. If lengths differ, the clip is padded or trimmed to the audio length.
- Frames are never dropped. That keeps visual frame t aligned with audio feature frames 4t to 4t plus 3.

BACKGROUND YOU SHOULD UNDERSTAND

Offline, a centred average is possible because the whole clip is available; live, only past frames exist, hence the exponential moving average.

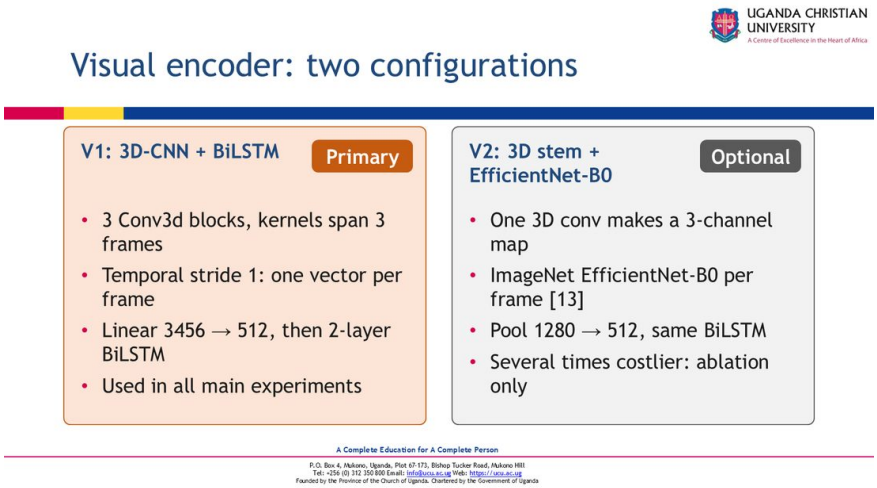
Padding repeats the last valid frame and marks it in a per-frame mask. In the live demo, a window with too many failures has its visual stream treated as missing (Part 4).

QUESTIONS YOU MAY BE ASKED

Why not just drop frames where the face is not found?

Dropping frames would break the fixed 4 to 1 alignment with audio. Interpolating and masking keeps both streams on the same time grid.

Slide 25 | Visual encoder: two configurations (1:00)



WHAT TO SAY

- The proposal names a 3D-CNN with a BiLSTM, and also EfficientNet-B0 as an efficient 2D backbone. These are two different ways to extract per-frame features, so we define two separate configurations instead of stacking both.
- V1, the primary one: three 3D convolution blocks whose kernels span three consecutive frames, so each feature responds to both mouth shape and how it changes. Temporal stride is 1, so we keep one vector per frame. Then a linear projection to 512 and a two-layer BiLSTM.
- V2, optional: one 3D convolution for short-term motion, then an ImageNet-pretrained EfficientNet-B0 on each frame, pooled to 1280 and projected to 512, then the same BiLSTM. It is several times more expensive, so it runs only as an ablation if time and compute allow.

BACKGROUND YOU SHOULD UNDERSTAND

A 3D convolution slides its kernel over time as well as height and width. A kernel spanning 3 frames can respond to motion such as 'lips closing', which no single frame shows.

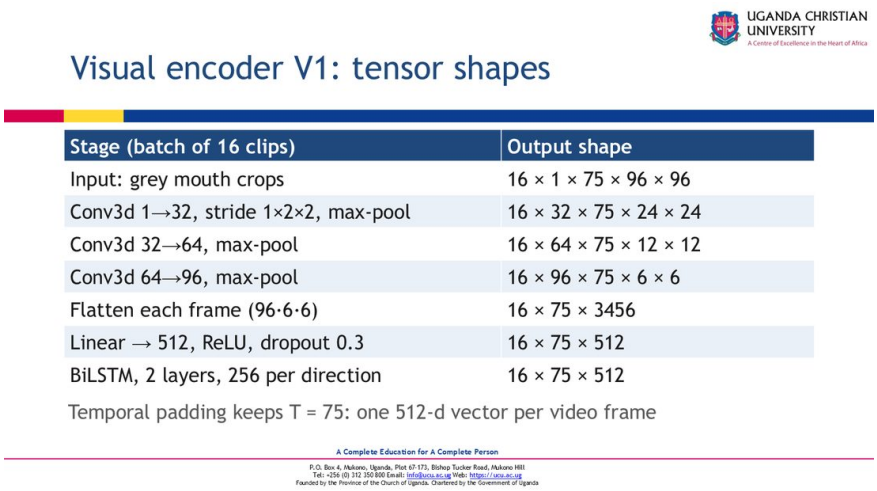
V2 folds time into the batch dimension so the 2D network processes each of the 75 frames independently. This mirrors the common '3D front-end plus 2D trunk' design [6] and tests whether transfer learning from a pretrained 2D backbone helps on GRID [5], [13].

QUESTIONS YOU MAY BE ASKED

Isn't transfer learning expected in this course?

Yes; V2 is our transfer-learning experiment (X4). V1 is primary because it is cheaper and is the proposal's '3D convolutional visual encoder' advanced technique.

Slide 26 | Visual encoder V1: tensor shapes (1:00)



WHAT TO SAY

- This traces one batch of 16 clips through V1.
- Input is 16 by 1 channel by 75 frames by 96 by 96. Block 1, a 3D convolution with spatial stride 2 plus max-pooling, gives 32 channels at 24 by 24. Block 2 gives 64 channels at 12 by 12. Block 3 gives 96 channels at 6 by 6.
- Each frame is then flattened, 96 times 6 times 6 is 3,456 values, projected to 512, and passed to a two-layer BiLSTM with 256 units per direction, giving 16 by 75 by 512.
- Temporal padding of one frame on each side keeps time at exactly 75, so we get one 512-dimensional vector per video frame.

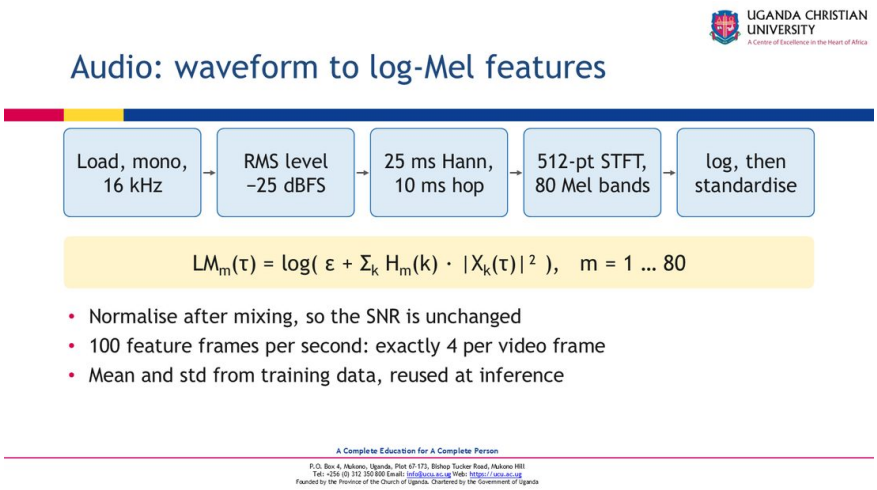
BACKGROUND YOU SHOULD UNDERSTAND

Shape convention for 3D convolutions in PyTorch: batch, channels, time, height, width.

Kernels: block 1 is 3x5x5 (time x height x width), block 2 is 3x5x5, block 3 is 3x3x3. Only spatial dimensions are downsampled; channel counts 32/64/96 are recommended starting values.

Testing: a random tensor of shape (2, 1, 75, 96, 96) must give (2, 75, 512) with finite values.

Slide 27 | Audio: waveform to log-Mel features (1:15)



WHAT TO SAY

- Audio features are computed on the fly because noise changes every time a clip is loaded.

- We load the waveform, average stereo to mono and resample to 16 kHz. We scale it to a fixed loudness of -25 dBFS, with gain capped at +20 dB so near-silence is not blown up into loud noise.
- The signal is cut into 25 ms frames every 10 ms, each multiplied by a Hann window. A 512-point Fourier transform gives the power spectrum, 80 Mel filters from 20 Hz to 8 kHz group it into bands, and we take the log. That is the formula in yellow.
- Normalisation happens after noise mixing, so the SNR is unchanged. The 10 ms hop gives 100 frames per second, exactly 4 per video frame. Feature means and deviations come from training data and are reused unchanged at inference.

BACKGROUND YOU SHOULD UNDERSTAND

dBFS = decibels relative to full scale; -25 dBFS is a comfortable fixed level that removes recording-level differences between GRID speakers and the demo microphone.

25 ms frames are used because speech is roughly stationary over 20 to 30 ms. The Hann window reduces spectral leakage at frame edges.

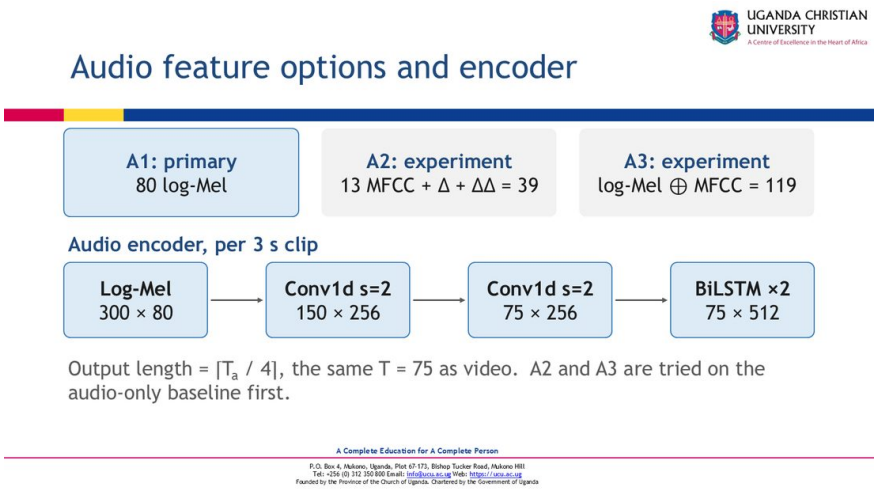
The Mel scale spaces filters more densely at low frequencies, like human pitch perception; the log compresses the large dynamic range of speech energy. Epsilon (1e-6) avoids log of zero.

QUESTIONS YOU MAY BE ASKED

Why compute features on the GPU rather than cache them?

Because the noise, and therefore the features, change every time a clip is loaded during training.

Slide 28 | Audio feature options and encoder (1:00)



WHAT TO SAY

- The proposal lists both log-Mel spectrograms and MFCCs. MFCCs are a linear transform of log-Mel, so a network given log-Mel can in principle learn anything MFCCs contain.
- So A1, 80 log-Mel features, is our primary configuration. A2, 13 MFCCs with deltas, 39 values, and A3, both combined, 119 values, are experiments, tried on the fast audio-only baseline first.
- The audio encoder: a 3 second clip gives 300 log-Mel frames. Two 1D convolutions with stride 2 bring that to 150 and then 75 frames with 256 channels, and a two-layer BiLSTM outputs 75 by 512, the same shape as the visual encoder.
- The output length is T_a over 4, rounded up. We use that formula to give CTC the true lengths.

BACKGROUND YOU SHOULD UNDERSTAND

MFCC: apply a discrete cosine transform to the log-Mel vector and keep the first 13 coefficients; deltas are first and second time derivatives.

Strided convolutions are preferred to simply discarding frames because they learn which local acoustic detail to keep. Length formula with kernel 3, padding 1, stride 2: floor((L + 2 - 3)/2) + 1 = ceil(L/2).

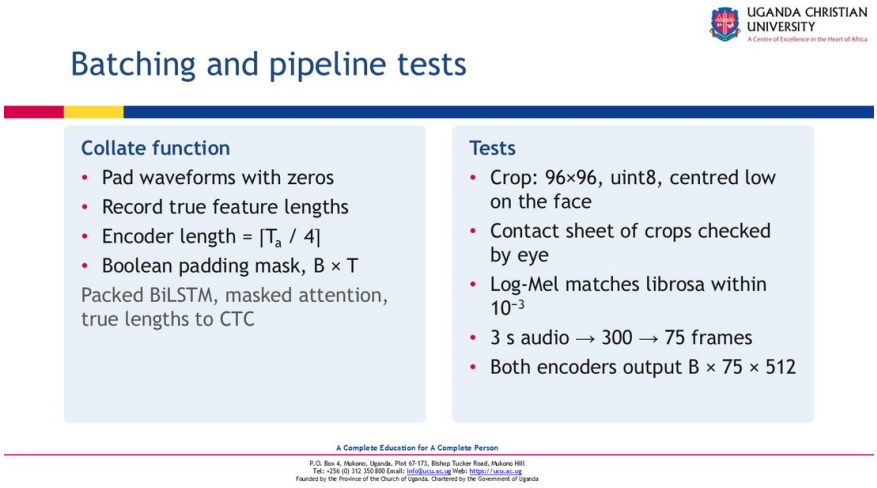
The STFT with centred frames gives 301 frames for 48,000 samples; the last is dropped so T_a = 300 = 4T.

QUESTIONS YOU MAY BE ASKED

Why not use MFCC and log-Mel together from the start?

It is a one-line configuration change, so we test it (A3) rather than assume it helps. We adopt it only if it improves validation WER.

Slide 29 | Batching and pipeline tests (1:00)



WHAT TO SAY

- Most GRID clips are 3 seconds, but some are shorter, and live windows vary. The collate function pads waveforms with zeros to the longest in the batch, records each item's true feature length, computes encoder lengths as T_a over 4 rounded up, and builds a padding mask.
- Those lengths are used three times: the BiLSTM uses packed sequences so padding does not leak into the backward direction, attention ignores padded positions through the mask, and CTC receives the true lengths.
- Tests for this part: the crop function returns 96 by 96 8-bit images centred low on the face; a contact sheet of crops from every speaker is checked by eye before training; our log-Mel output matches librosa within 10 to the minus 3; 3 seconds of audio gives exactly 300 feature frames and 75 encoder frames; and both encoders output batch by 75 by 512.
- Lisa will now explain the multimodal model.

BACKGROUND YOU SHOULD UNDERSTAND

Passing padded lengths instead of true lengths to CTC is a common silent bug; it does not crash, it just trains worse.

Clips of similar length can be grouped into batches (bucketing) to reduce wasted computation on padding.

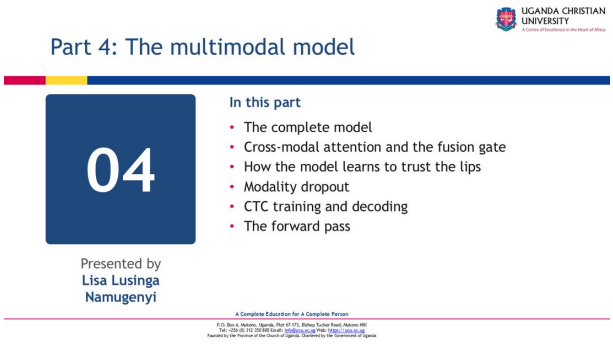
Normalised training features must also have per-band mean within 0.05 of 0 and standard deviation within 0.05 of 1.

Part 4: The multimodal model

Presenter: **Lisa Lusinga Namugenyi**

Slides 30–38 | suggested total 8:40

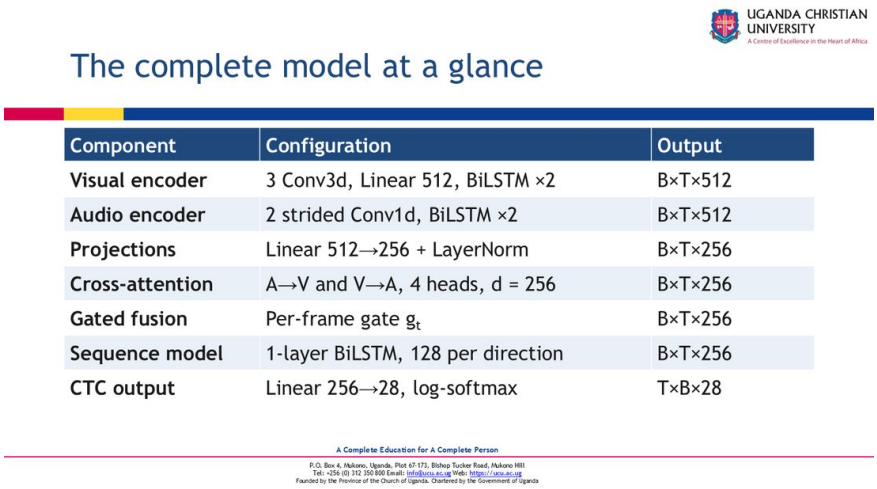
Slide 30 | Part 4: The multimodal model (0:10)



WHAT TO SAY

- Part 4: the complete model, how the streams are fused, and how we train and decode it.

Slide 31 | The complete model at a glance (0:45)



WHAT TO SAY

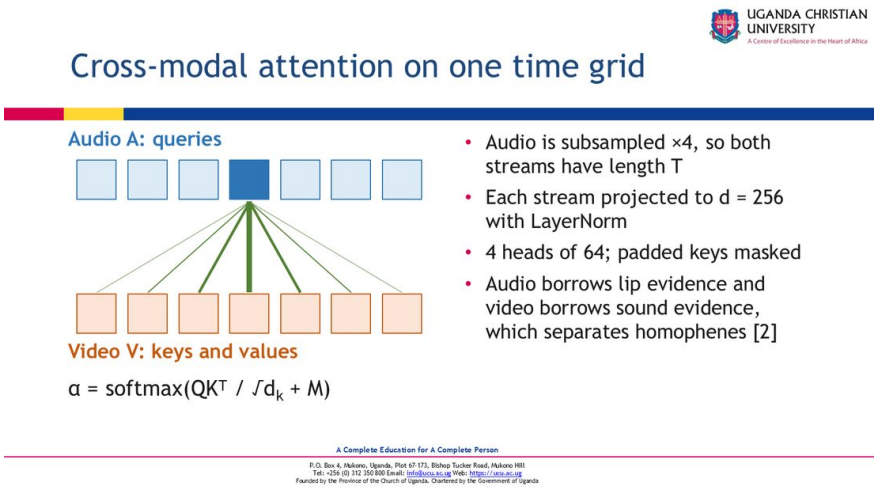
- This is the whole model on one table.
- The visual and audio encoders each output batch by T by 512. Each is projected to 256 with layer normalisation. Cross-attention runs in both directions with 4 heads. A per-frame gate combines the two. A one-layer BiLSTM with 128 units per direction models the fused sequence, and a linear layer with log-softmax gives 28 symbol probabilities per frame.
- All of these sizes are recommended starting values that we will tune.

BACKGROUND YOU SHOULD UNDERSTAND

Output of the CTC layer is time-first (T x B x 28) because that is the layout PyTorch's CTCLoss expects.

The visual-only and audio-only baselines use the same encoders and the same sequence BiLSTM and CTC head, without fusion.

Slide 32 | Cross-modal attention on one time grid (1:15)



WHAT TO SAY

- Because the audio encoder subsamples by exactly 4, both encoders emit sequences of the same length T on the same 40 ms grid. If rounding at clip edges makes them differ by a frame or two, the longer one is trimmed.
- Each stream is projected to 256 dimensions with layer normalisation. Then each stream queries the other. In the picture, one audio frame, the dark blue square, is the query; the video frames are keys and values. The line widths show attention weights: it draws mostly on the matching video frame and its neighbours.
- Weights are a softmax of queries times keys over the square root of the key size, with a mask M that blocks padded positions. We use 4 heads of 64 dimensions.
- The effect: audio can borrow lip evidence for an ambiguous sound, and video can borrow sound evidence for an ambiguous lip shape. That is the mechanism for separating homophenes [2].

BACKGROUND YOU SHOULD UNDERSTAND

After attention, each stream adds the attended result back and normalises: A-tilde = LN(A + MHA(A, V)) and V-tilde = LN(V + MHA(V, A)). This is a residual connection.

Cross-attention itself does not need equal lengths; the per-frame gate on the next slide does.

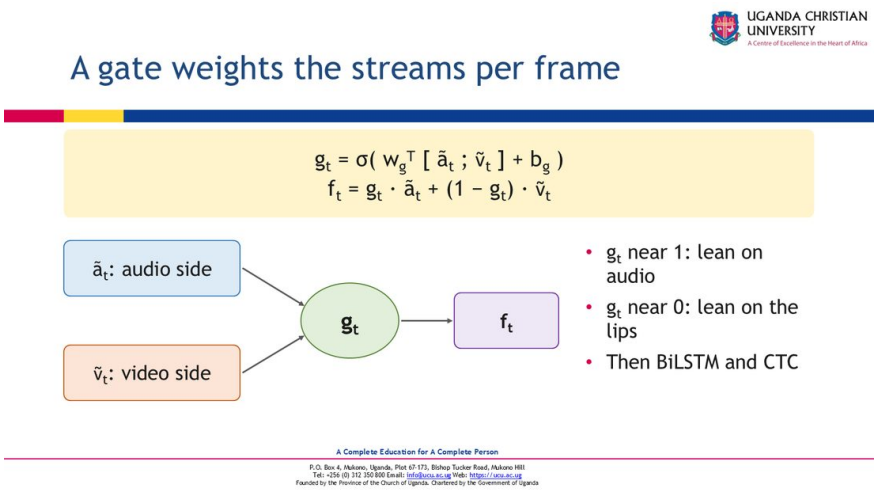
Implemented with nn.MultiheadAttention. The line widths in the diagram are illustrative, not measured.

QUESTIONS YOU MAY BE ASKED

What is the difference between self-attention and cross-modal attention?

In self-attention, queries, keys and values all come from the same sequence. In cross-modal attention, queries come from one modality and keys and values from the other.

Slide 33 | A gate weights the streams per frame (1:00)



WHAT TO SAY

- After cross-attention we have an audio-anchored vector a-tilde and a video-anchored vector v-tilde for every frame.
- A learned gate g_t, a sigmoid of a weighted combination of both, decides frame by frame how much each contributes. The fused vector f_t is g times the audio side plus 1 minus g times the video side.
- A g close to 1 leans on audio; close to 0 leans on the lips. The fused sequence then goes through the BiLSTM and the CTC layer.

BACKGROUND YOU SHOULD UNDERSTAND

Sigma is the logistic function, which squashes any number into the range 0 to 1. The square brackets mean the two vectors are concatenated.

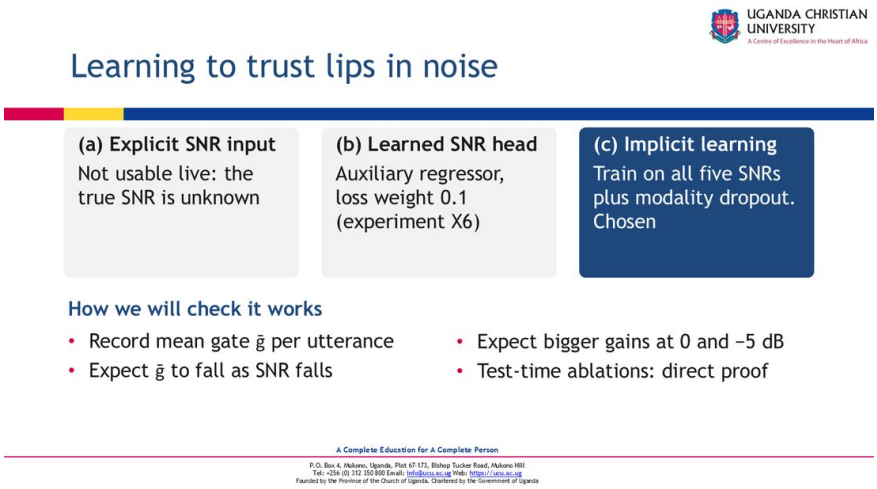
Because a-tilde already contains some visual information through cross-attention, g_t is only an approximate measure of reliance on audio. The test-time modality ablations in Part 5 give the more direct evidence.

QUESTIONS YOU MAY BE ASKED

Is the gate one number per frame or one per feature?

One scalar per frame, in the range 0 to 1.

Slide 34 | Learning to trust lips in noise (1:00)



WHAT TO SAY

- The proposal says fusion should give more weight to the lips when the audio is noisy. But attention does not detect noise by itself; it learns whatever reduces the training loss.
- There are three ways to get that behaviour. (a) Feed the true SNR to the gate: not usable live, because the true SNR of room audio is unknown. (b) Train a small head to estimate the SNR from the audio and feed that in: kept as experiment X6. (c) Implicit learning: train on all five SNRs plus modality dropout, so the gate sees many cases where audio is useless or absent. That is our choice.
- To check it works, we record the mean gate value for every test clip. If it works, the mean gate should fall as SNR falls, and the gain over audio-only should grow at 0 and -5 dB. Test-time ablations give the direct proof.

BACKGROUND YOU SHOULD UNDERSTAND

Option (b) details: an auxiliary linear head on the time-averaged audio encoding regresses the known training SNR (clean mapped to 30 dB); its prediction is appended to the gate input, and its loss is added to the CTC loss with weight about 0.1.

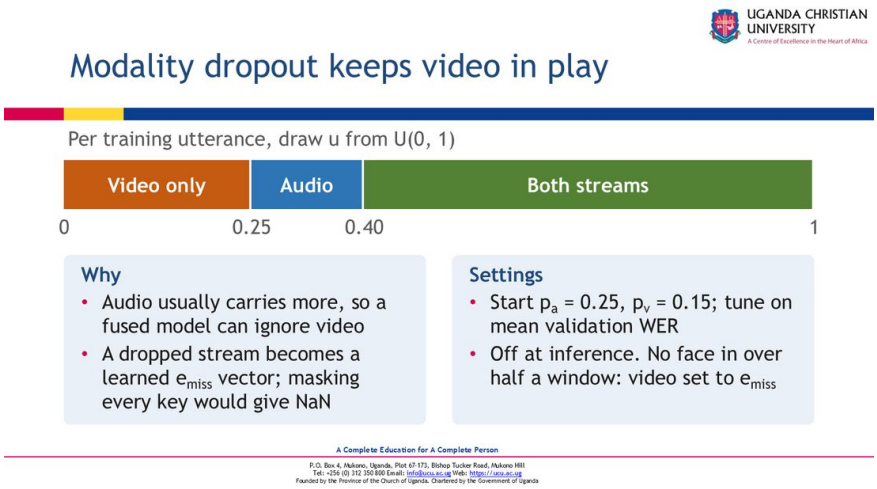
Test-time ablations: replace the visual encoding by the 'missing' vector (audio-only operation) or the audio encoding by it (visual-only operation), without retraining.

QUESTIONS YOU MAY BE ASKED

What if the gate does not change with SNR?

That is a finding, and we would report it. Nehum's diagnosis checklist in Part 5 covers what we would check next.

Slide 35 | Modality dropout keeps video in play (1:15)



WHAT TO SAY

- Audio is usually far more informative than video, so a fused model can learn to ignore the video entirely. Modality dropout prevents that by sometimes removing one stream during training [6].
- For each training clip we draw a random number u. Below 0.25, the audio is dropped and the model must use video only. Between 0.25 and 0.40, the video is dropped. Otherwise both streams are used. Both are never dropped together.
- A dropped stream is replaced by a learned 'missing modality' vector, e-miss, not by masking. Masking every key would make the attention softmax undefined and produce NaNs.
- We start at p_a 0.25 and p_v 0.15 and tune on mean validation WER. At inference it is off; but if no face is found in more than half of a window, the video is replaced by e-miss, which the model has seen in training, so it falls back to audio-only cleanly.

BACKGROUND YOU SHOULD UNDERSTAND

Audio is dropped more often than video because the risk being addressed is under-use of video.

Tuning grid if compute allows: p_a and p_v in {0, 0.1, 0.25, 0.4} with p_a + p_v at most 0.5. Setting both to 0 is ablation X1, 'no modality dropout'.

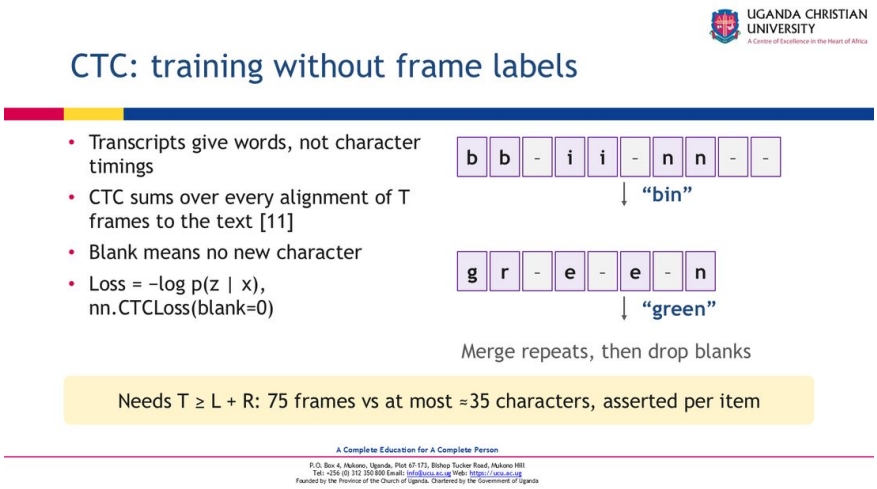
Why masking fails: softmax over a row where every logit is minus infinity is 0/0, which is NaN, and NaN then spreads through the network.

QUESTIONS YOU MAY BE ASKED

Why a learned vector and not zeros?

A learned vector lets the gate recognise 'this stream is missing' as a distinct state. Zeros could look like a legitimate, if odd, input.

Slide 36 | CTC: training without frame labels (1:15)



WHAT TO SAY

- Our transcripts say which words were spoken, but not at which frame each character occurs. Connectionist Temporal Classification, CTC [11], removes the need for frame labels by summing over every possible alignment of the T output frames to the characters.
- It adds a blank symbol that means 'no new character'. To read out text, merge repeated symbols, then remove blanks. So b b blank i i blank n n blank blank becomes 'bin'. A doubled letter needs a blank between its copies, as in g r blank e blank e blank n, which becomes 'green'.
- The loss is minus the log probability of the correct transcript, computed efficiently by PyTorch's CTCLoss with blank index 0 and zero_infinity on.
- CTC needs enough frames: T must be at least the transcript length plus the number of repeated letters. A GRID transcript is at most about 35 characters and we have 75 frames, so there is a wide margin. The data loader asserts this for every item.

BACKGROUND YOU SHOULD UNDERSTAND

zero_infinity=True means an occasional impossible alignment gives zero loss instead of infinity, which would otherwise corrupt training.

The condition would fail if the audio encoder subsampled more aggressively or if live windows were very short; that is why the live window is 3 s.

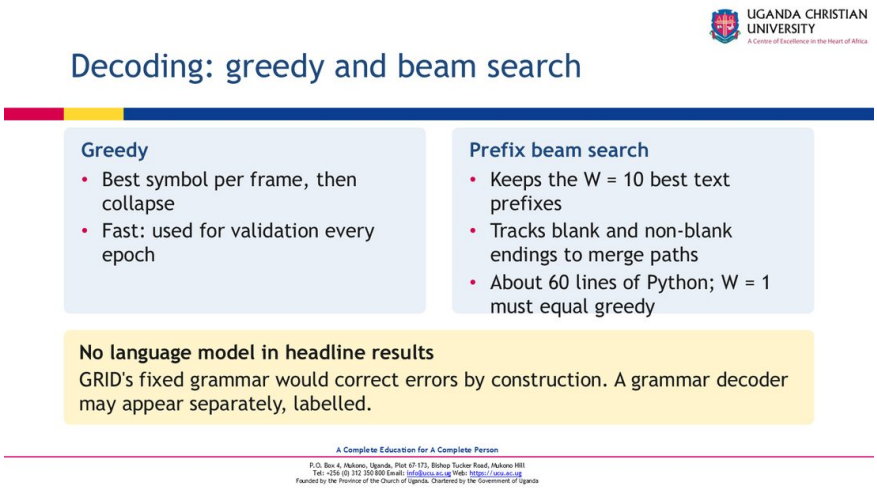
CTC is used by the CTC-based lipreading and AVSR systems in our references [4], [6].

QUESTIONS YOU MAY BE ASKED

Why not an attention-based decoder like in transformers?

CTC is simpler, trains reliably with small data, matches the systems we cite, and its alignment assumption suits a monotonic task like speech, where text order follows time order.

Slide 37 | Decoding: greedy and beam search (1:00)



WHAT TO SAY

- Two decoders. Greedy decoding takes the most probable symbol at each frame and collapses. It is fast, so we use it for validation every epoch.
- For final results and the demo, the proposal specifies beam search. CTC prefix beam search keeps the 10 most probable text prefixes, tracking separately the probability of ending in a blank and not, so paths that collapse to the same text are merged. With only 28 symbols this is about 60 lines of Python, and a test checks that beam width 1 equals greedy.
- We will not use a language model for headline results. GRID's fixed six-slot grammar would correct many errors by construction, which would not carry over to real captioning. A grammar-constrained decoder may be shown separately, clearly labelled, as an upper reference.

BACKGROUND YOU SHOULD UNDERSTAND

Why track blank and non-blank endings: a repeated letter after a blank starts a new character, but a repeated letter without a blank collapses into the same one. The two cases must be scored separately.

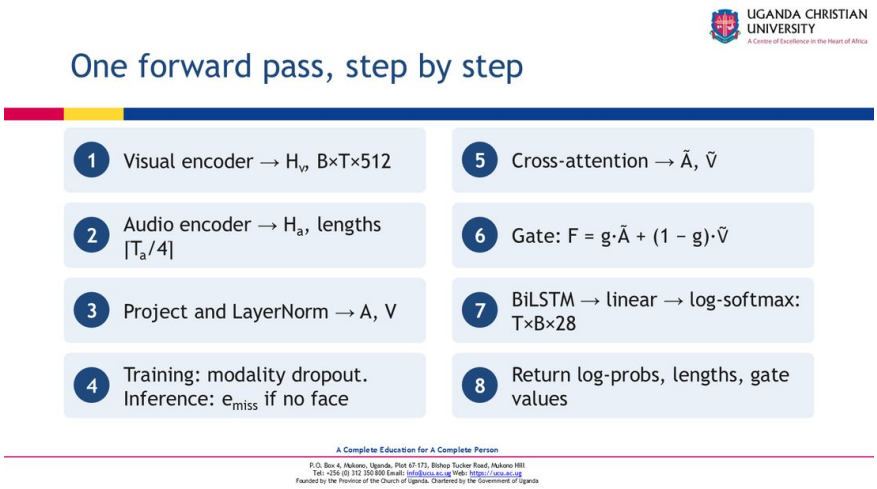
The beam-search algorithm in the implementation document gives the full pseudocode.

QUESTIONS YOU MAY BE ASKED

Isn't refusing a language model leaving accuracy on the table?

On GRID it would inflate accuracy in a way that says nothing about real use. We prefer honest headline numbers and can show the grammar decoder separately.

Slide 38 | One forward pass, step by step (1:00)



WHAT TO SAY

- To pull it together, one forward pass.
- One and two: the visual and audio encoders produce H_v and H_a, both batch by T by 512, with encoder lengths from the audio. Three: project both and normalise. Four: in training apply modality dropout; at inference, replace video with e-miss if the face is missing.
- Five: cross-attention in both directions. Six: the gate mixes the two. Seven: BiLSTM, linear and log-softmax, giving T by batch by 28. Eight: return the log-probabilities, the lengths for CTC, and the gate values, which we log for analysis.
- Nehum will now explain how we train and evaluate this model.

BACKGROUND YOU SHOULD UNDERSTAND

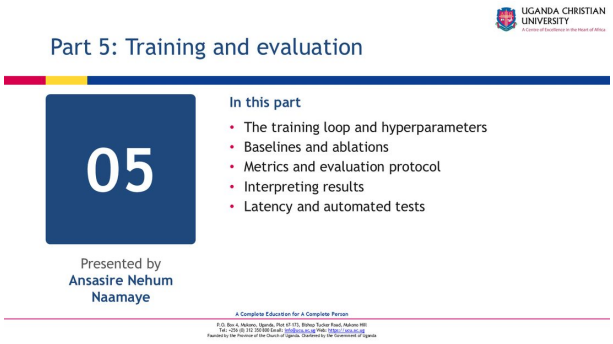
This is the forward-pass algorithm in the implementation document. The same function runs in training and inference; only step 4 and the surrounding loss or decoder differ.

Part 5: Training and evaluation

Presenter: **Ansasire Nehum Naamaye**

Slides 39–47 | suggested total 8:55

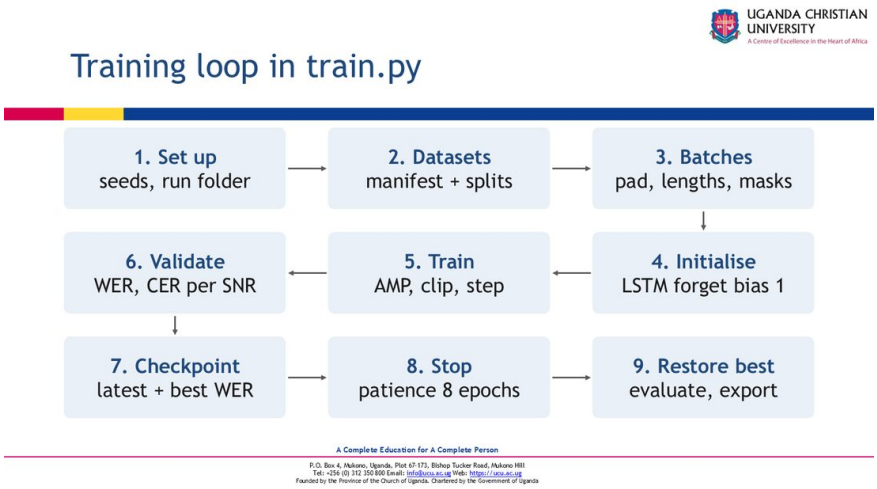
Slide 39 | Part 5: Training and evaluation (0:10)



WHAT TO SAY

- Part 5: how we train, what we compare against, and how we measure success.

Slide 40 | Training loop in train.py (1:15)



WHAT TO SAY

- Training is driven by one script, train.py, reading one YAML config. It has nine steps.
- Set up: load the config, set all seeds, create a run folder and save the config and commit hash. Build datasets from the manifest and the split file. Build batches with padding, lengths and masks; log-Mel features are computed on the GPU here.
- Initialise with PyTorch defaults, with LSTM forget-gate bias set to 1 to help gradients early on. Train with mixed precision, gradient clipping, and optimiser and scheduler steps. Validate after each epoch: loss, WER and CER at every SNR and their mean.
- Checkpoint the latest state and, separately, the best by mean validation WER. Stop when WER has not improved for 8 epochs or at 60 epochs. Finally restore the best checkpoint, run the final evaluation and export the weights for the demo.

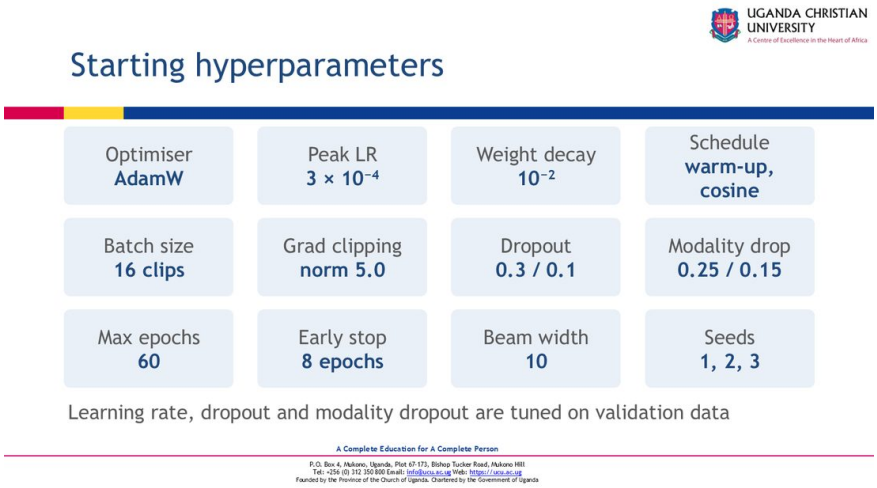
BACKGROUND YOU SHOULD UNDERSTAND

A checkpoint contains model, optimiser, scheduler and mixed-precision scaler states, the epoch, the config and the vocabulary, so a run can be resumed exactly.

Forget-gate bias of 1 makes LSTMs remember by default at the start of training, which eases gradient flow.

If a batch gives a non-finite loss, the step is skipped and logged; more than a handful indicates a data or length bug.

Slide 41 | Starting hyperparameters (1:00)



WHAT TO SAY

- These are our starting values. They are common defaults for models of this size, not proven optima for LipScribe.
- AdamW optimiser, peak learning rate 3 times 10 to the minus 4, weight decay 10 to the minus 2, with one epoch of linear warm-up then cosine decay. Batch of 16 clips. Gradients clipped at norm 5.
- Dropout 0.3 in the encoders and 0.1 in attention; modality dropout 0.25 and 0.15. Up to 60 epochs with early stopping after 8 without improvement. Beam width 10. Seeds 1, 2 and 3 for the final models if compute allows.
- The most sensitive values, learning rate, dropout and modality dropout, are tuned on validation data only.

BACKGROUND YOU SHOULD UNDERSTAND

Learning rate is tuned over {1e-4, 3e-4, 1e-3}. Warm-up stabilises early CTC training, which is known to be fragile.

If batch 16 does not fit in GPU memory, gradient accumulation gives the same effective batch size over several smaller steps.

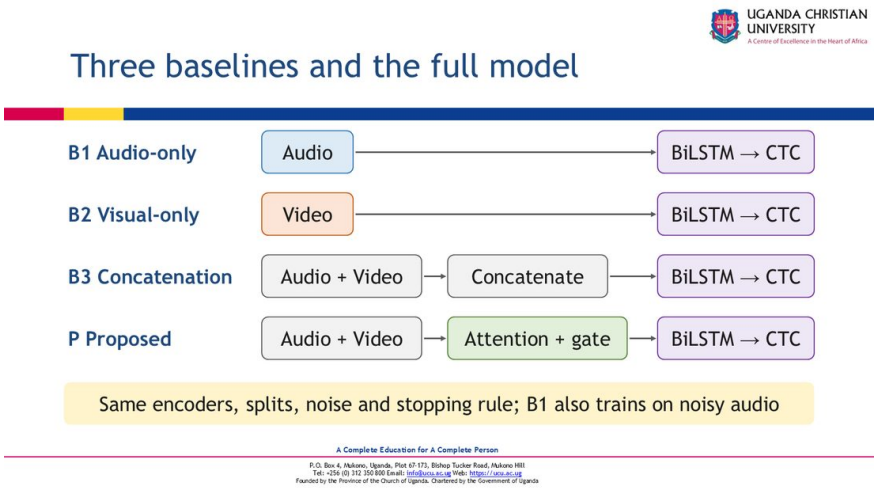
Curriculum (to be decided): if the visual-only model is slow to start learning, we first train the visual encoder alone and use it to initialise the fused model's visual stream.

QUESTIONS YOU MAY BE ASKED

Why AdamW rather than Adam?

AdamW applies weight decay correctly, separately from the adaptive gradient scaling. It is a robust default for recurrent and attention models.

Slide 42 | Three baselines and the full model (1:00)



WHAT TO SAY

- The proposal requires three simple baselines and then the full system.
- B1, audio-only: audio encoder, BiLSTM, CTC. B2, visual-only: visual encoder, BiLSTM, CTC. B3, simple fusion: concatenate the projected audio and video per frame, project to 256, then the same BiLSTM and CTC, with no attention and no modality dropout. P, our proposed model, uses attention and the gate, with modality dropout.
- Everything else is shared: encoders, splits, noise files, normalisation, optimiser, schedule and stopping rule. Only the component under test differs.
- Importantly, the audio-only baseline also trains on noisy audio. A baseline trained only on clean audio would be unfairly weak.

BACKGROUND YOU SHOULD UNDERSTAND

B1, B2, B3 and P form the core and are completed first.

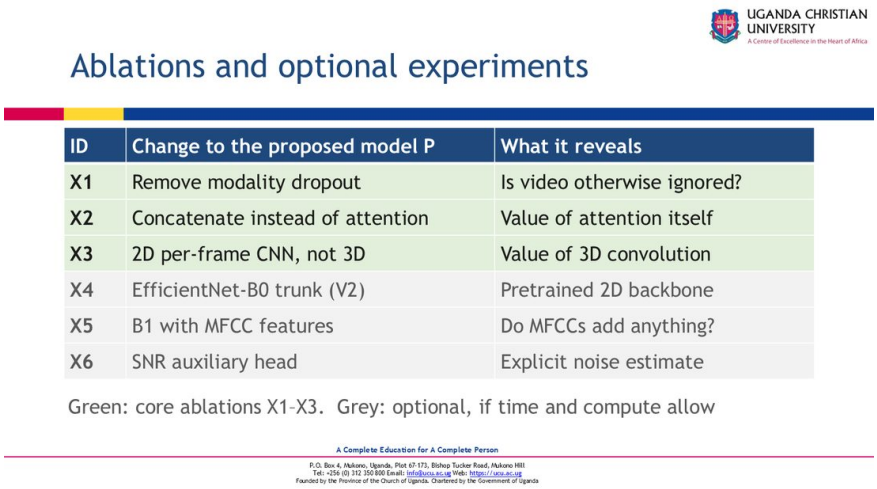
B2 is constant across SNR by design (no audio), so it is an upper bound on what the lips alone give on GRID and a check that the visual pipeline works.

QUESTIONS YOU MAY BE ASKED

Why is B3 needed if you already have B1 and P?

It separates two questions: does adding video help at all (B3 vs B1), and does adaptive fusion help beyond naive combination (P vs B3)?

Slide 43 | Ablations and optional experiments (1:00)



WHAT TO SAY

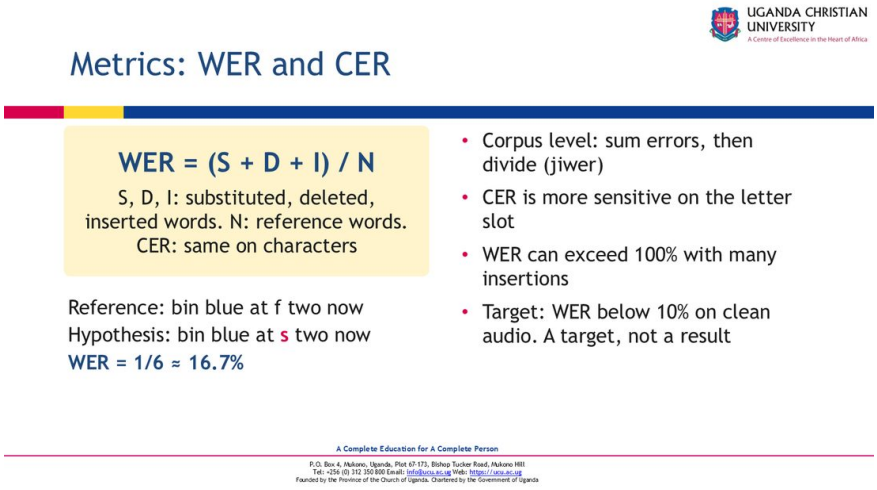
- Ablations each remove one advanced component and keep everything else fixed.
- X1 removes modality dropout: does the model then ignore video? X2 replaces attention with concatenation but keeps dropout: what does attention itself add? X3 replaces the 3D front-end with a 2D per-frame CNN: what does spatiotemporal convolution add? These three are core and are run right after the baselines.
- X4 to X6 are optional: the pretrained EfficientNet-B0 trunk, MFCC features on the audio baseline, and the SNR auxiliary head. They depend on remaining time and compute.

BACKGROUND YOU SHOULD UNDERSTAND

X1 to X3 are the ablations the proposal asks for. Each is evaluated with the same per-SNR protocol as P.

Two extra test-time ablations need no retraining: run P with the video replaced by e-miss (audio-only operation) or with the audio replaced (visual-only operation).

Slide 44 | Metrics: WER and CER (1:00)



WHAT TO SAY

- The proposal specifies word error rate and character error rate.
- WER is substitutions plus deletions plus insertions, divided by the number of reference words, from the minimum edit-distance alignment. CER is the same on characters, including spaces.

- Example from our test suite: reference 'bin blue at f two now', hypothesis 'bin blue at s two now'. One substitution in six words: WER is 1 in 6, about 16.7%.
- We compute at corpus level: sum errors over the whole test set, then divide, as jiwer does. CER is more sensitive on GRID's letter slot, where one wrong character makes a whole word wrong. WER can exceed 100% if there are many insertions. Our target is WER below 10% on clean audio; that is a target to test, not a result.

BACKGROUND YOU SHOULD UNDERSTAND

Averaging per-utterance rates would give short sentences too much weight; corpus-level is the standard convention.

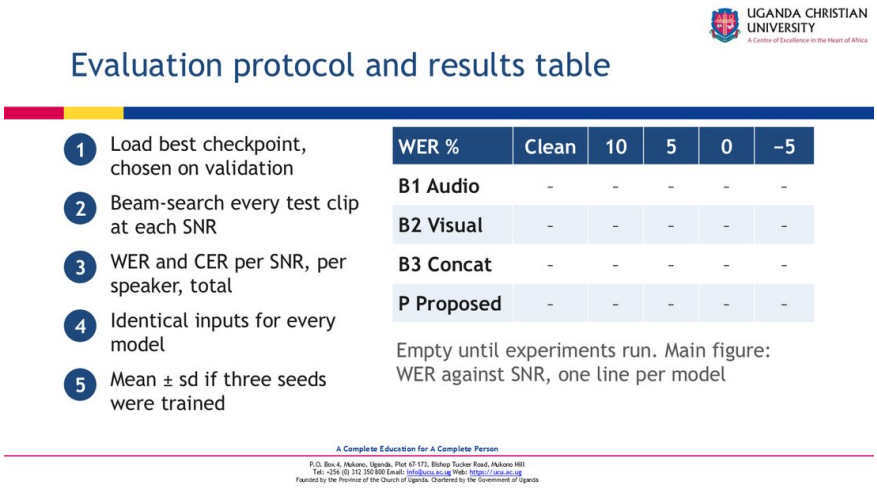
The letter slot is the hardest in GRID: letters such as 'b', 'p' and 'm' look alike on the lips, and 'f' and 's' sound alike in noise.

QUESTIONS YOU MAY BE ASKED

How does 10% compare with the published 2.7% clean WER from the proposal?

That 2.7% was on LRS2, a different dataset. Never compare numbers across datasets. Our 10% is a project target on GRID.

Slide 45 | Evaluation protocol and results table (1:00)



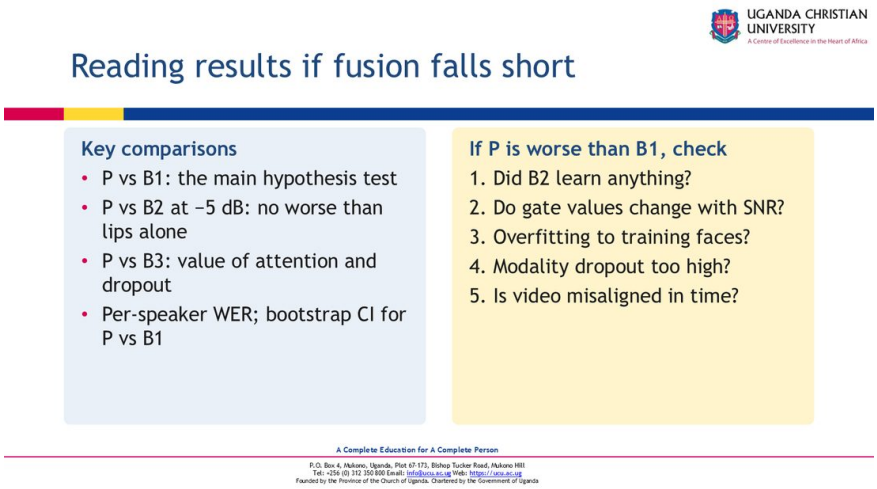
WHAT TO SAY

- The protocol has five steps. Load each model's best checkpoint, chosen on validation data only. Decode every test clip of the four held-out speakers with beam search, at each of the five SNRs, using the fixed pre-mixed audio. Compute corpus WER and CER per SNR, per speaker and overall, and record the mean gate per clip. Use identical inputs for every model. Where three seeds were trained, report mean and standard deviation.
- We report each SNR separately, because averaging across noise levels would hide exactly what the project measures: how fast accuracy degrades and where video starts to help.
- The table on the right is the template for our main result. It is empty on purpose: no results exist yet. The main figure will be WER against SNR with one line per model.

BACKGROUND YOU SHOULD UNDERSTAND

The same empty template is in the implementation document and will be filled only from logged runs.

Slide 46 | Reading results if fusion falls short (1:15)



WHAT TO SAY

- Key comparisons. P against B1 is the main test of our hypothesis. P against B2 at -5 dB: the fused model should be no worse than lips alone, since in the limit it can ignore the audio; if it is worse, noisy audio is misleading the fusion. P against B3 isolates the value of attention and modality dropout.
- All numbers are on unseen speakers, and we also report per-speaker WER, because a mean can hide one speaker the system fails on. With only four test speakers, small differences may not be meaningful, so we use a paired bootstrap to give a confidence interval for P against B1 at each SNR.
- If P is worse than audio-only, we check in order: did B2 learn anything at all; do gate values change with SNR; is the model overfitting to training faces; is modality dropout so high it starves the audio; and is the video misaligned in time.
- A fused model slightly worse on clean audio but better at low SNR is still consistent with our goal, and we will report it that way.

BACKGROUND YOU SHOULD UNDERSTAND

Paired bootstrap: resample the test utterances with replacement many times, compute the WER difference between two models on each resample, and read off the spread. 'Paired' means both models are scored on the same resampled utterances.

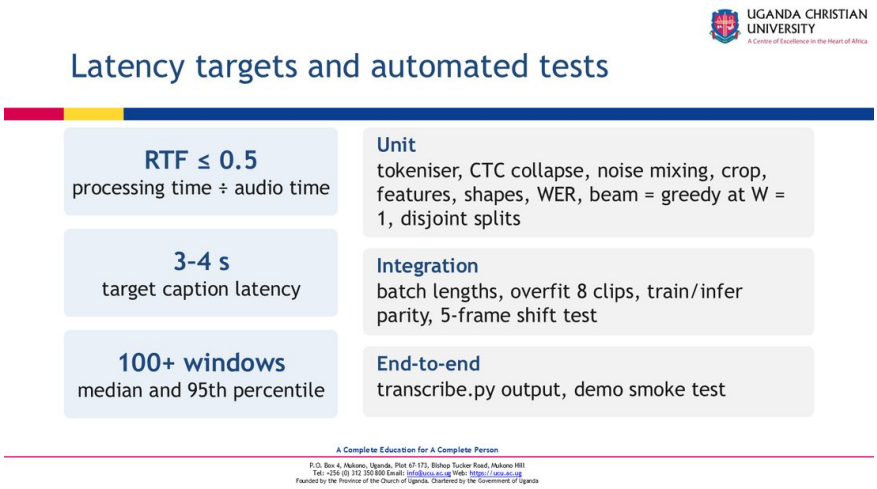
If B2 is near chance, the visual pipeline is broken and fusion cannot help; that is why it is the first check.

QUESTIONS YOU MAY BE ASKED

What if fusion never helps?

Then that is our result, reported honestly with the diagnosis. B3 is still a working fused system, and the per-SNR analysis still tells us where video does or does not help.

Slide 47 | Latency targets and automated tests (1:15)



WHAT TO SAY

- For the demo we measure, on the demo laptop, Face Mesh time per frame, model and beam-search time per window, end-to-end caption latency, and the real-time factor: processing time divided by audio duration.
- Live operation needs an RTF below 1; our target is 0.5 or less, with caption latency around 3 to 4 seconds. We report the median and 95th percentile over at least 100 windows after warm-up.
- Software tests are written with pytest and run before every merge, using small fixture files so they run without the full dataset. Unit tests cover the tokeniser, CTC collapse, noise mixing, cropping, features, encoder shapes, WER, beam search and disjoint splits. Integration tests cover batch lengths, overfitting 8 clips, train/inference parity, and a sync test: shifting video by 5 frames must make the trained model worse. End-to-end tests run transcribe.py and the demo.
- Isabelle will now present the application, the plan and the risks.

BACKGROUND YOU SHOULD UNDERSTAND

Overfit check: each model must reach CTC loss below 0.05 and CER of 0 on 8 utterances within 300 steps. If a model cannot memorise 8 clips, something is broken.

The 5-frame shift test is a sanity check of alignment: if shifting the video does not hurt, the model is not using the video's timing.

QUESTIONS YOU MAY BE ASKED

Why 95th percentile and not just the average?

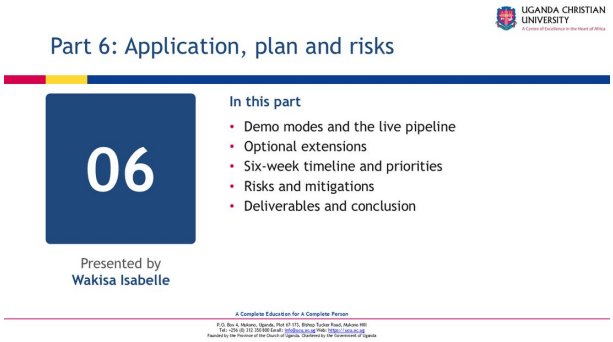
Users notice the slow cases. The 95th percentile shows how bad the occasional slow caption is.

Part 6: Application, plan and risks

Presenter: **Wakisa Isabelle**

Slides 48–61 | suggested total 10:05

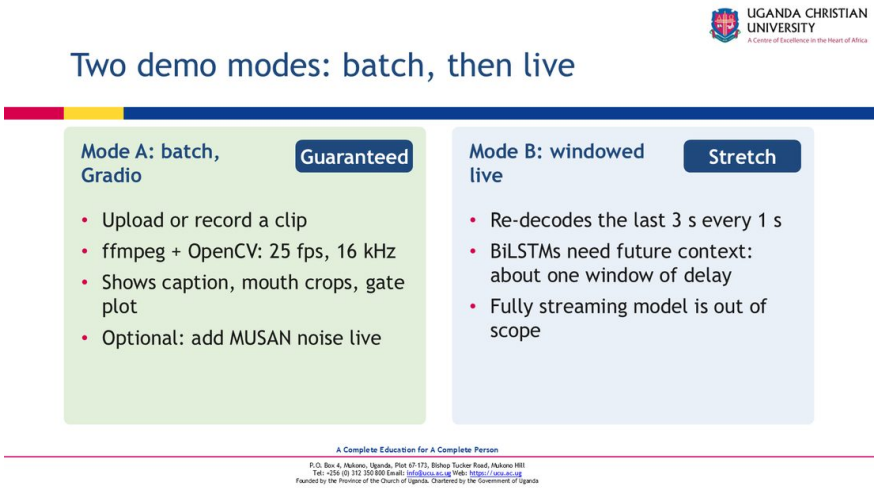
Slide 48 | Part 6: Application, plan and risks (0:10)



WHAT TO SAY

- Part 6: the demonstration, the extensions, the schedule, the risks, and what we will deliver.

Slide 49 | Two demo modes: batch, then live (1:00)



WHAT TO SAY

- We plan two demo modes.
- Mode A is batch captioning in Gradio, and it is our guaranteed deliverable. You upload or record a clip; the server extracts frames and audio with OpenCV and ffmpeg, resamples to 25 fps and 16 kHz, runs the pipelines, the model and beam search, and shows the caption next to the video, with the mouth crops and a plot of the fusion gate. An optional control adds MUSAN noise at a chosen SNR, so the audience can see captions with and without noise on the same recording.
- Mode B is windowed live captioning, our stretch goal. Because our encoders are bidirectional, they need future context, so we approximate streaming by re-decoding the latest 3 second window every second. That costs about one window of latency. A fully streaming model is outside our six-week scope.

BACKGROUND YOU SHOULD UNDERSTAND

Mode A reuses the evaluation code almost unchanged, so its correctness is easy to verify.

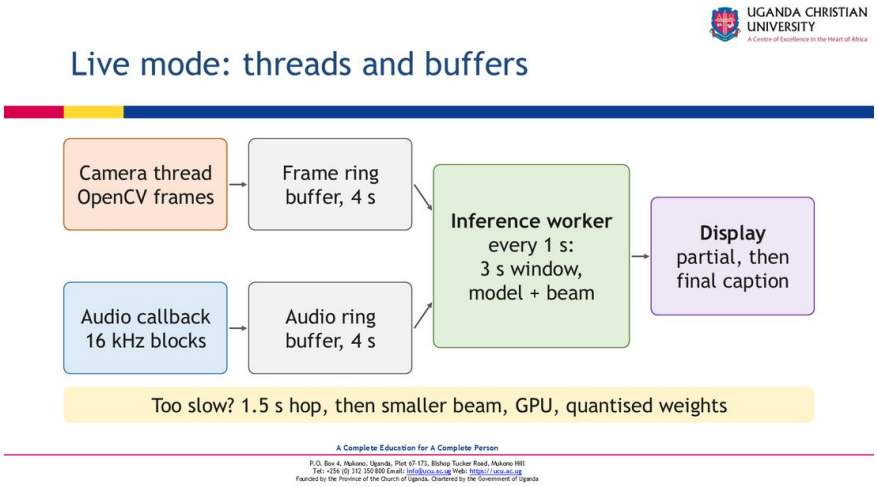
A fully streaming model would need unidirectional encoders and a different decoder; that is future work.

QUESTIONS YOU MAY BE ASKED

Is Mode B 'real-time'?

It is near real-time: captions appear about a window behind the speech. We measure the actual latency and report it.

Slide 50 | Live mode: threads and buffers (1:00)



WHAT TO SAY

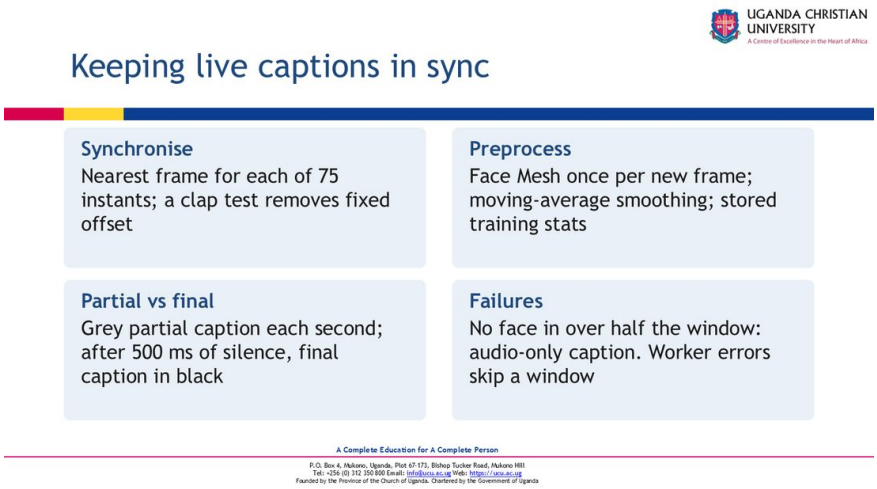
- This is Mode B's structure.
- A camera thread reads frames with OpenCV and timestamps each one. An audio callback appends 16 kHz blocks, also timestamped from the same clock. Each goes into a ring buffer holding the last 4 seconds.
- Every second, the inference worker cuts the latest 3 second window from both buffers, crops, computes features, runs the model and beam search, and sends text to the display, which shows a partial caption and then a final one.
- If it is too slow, we act in this order: process every 1.5 seconds instead of 1, reduce the beam width, use a GPU, or quantise the model weights.

BACKGROUND YOU SHOULD UNDERSTAND

Ring buffer: a fixed-size buffer that overwrites the oldest data, so memory stays bounded. Four seconds of 640x480 colour frames at 30 fps is about 110 MB; storing downscaled frames or precomputed crops reduces this.

The 3 s window matches GRID training clips and gives T = 75, which satisfies the CTC length condition.

Slide 51 | Keeping live captions in sync (1:00)



WHAT TO SAY

- Four details make the live mode work.
- Synchronise: webcams rarely deliver exactly 25 fps and may drop frames, so for each of the 75 target instants in a window we take the buffered frame with the nearest timestamp. A constant camera-to-microphone offset is measured once with a hand-clap test and subtracted.
- Preprocess: Face Mesh runs once on each new frame as it arrives, not once per window, and crop smoothing uses an exponential moving average. Audio uses the stored training statistics.
- Partial versus final: a simple voice-activity detector watches the audio. While speech continues, a grey partial caption is replaced every second. After at least 500 ms of silence, one window aligned to the utterance is decoded and shown as a final caption in black.
- Failures: if no face is found in over half the window, the caption is marked audio only. If the camera or microphone cannot be opened, the app continues with the other where possible. Errors in the worker are logged and that window is skipped.

BACKGROUND YOU SHOULD UNDERSTAND

Clap test: the frame where the hands meet should line up with the audio onset of the clap; the difference is the offset.

Restricting final captions to detected utterances avoids the hard problem of merging text from overlapping windows.

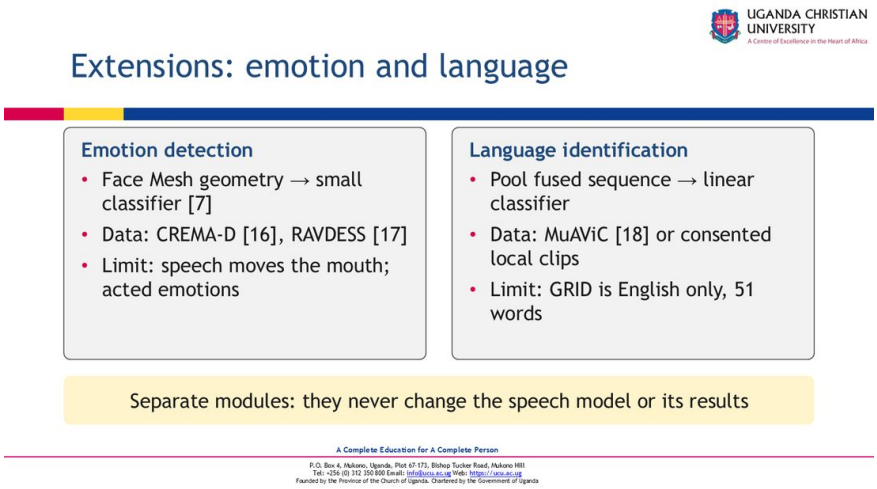
Testing the app: held-out GRID clips fed through Mode A and Mode B must give the same captions as the evaluation script. Live tests use team members reading GRID-grammar sentences, in quiet and with played-back noise. Covering the mouth or leaving the frame must give 'audio only' captions, not crashes.

QUESTIONS YOU MAY BE ASKED

Will it work if I just talk normally to it?

Not reliably. The model is trained on GRID's fixed grammar and framing. The demo uses GRID-style sentences and we say so openly.

Slide 52 | Extensions: emotion and language (1:00)



WHAT TO SAY

- The two extension objectives are separate, optional modules.
- Emotion detection would reuse the Face Mesh landmarks, compute geometric features such as mouth opening, eyebrow-to-eye distance and eye aperture after removing head pose, summarise them over the clip, and classify with a small model, following Siam et al. [7]. Data would come from emotion-labelled sets such as CREMA-D or RAVDESS. The limits: while speaking, the mouth mostly reflects what is said, and acted emotions are exaggerated.
- Language identification would pool the fused sequence over time and add a linear classifier over candidate languages. It needs multilingual labelled data such as MuAViC, or locally recorded phrases with consent. The limit: GRID is English only with 51 words.
- Neither changes the speech model or its results.

BACKGROUND YOU SHOULD UNDERSTAND

Siam et al. report up to 97% emotion accuracy on their data; we do not claim that figure for our setting.

Facial expression is not a reliable indicator of internal emotional state, and expressions vary across people and cultures. Any emotion output would be shown as an experimental label, not a statement about the user.

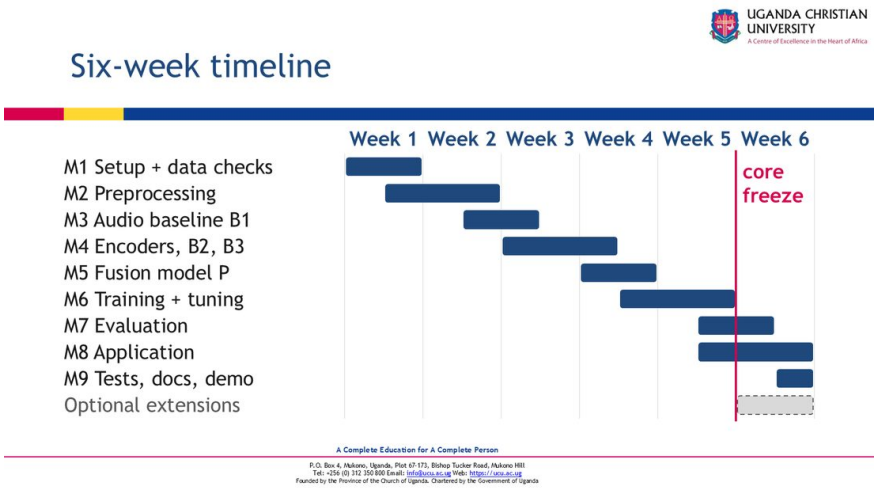
Language ID on short clips can be confounded by speaker identity if each language is spoken by different people, so speaker-disjoint evaluation is again required.

QUESTIONS YOU MAY BE ASKED

Could LipScribe work in a Ugandan language?

Not as trained. It would need transcribed audio-visual data in that language and an extended character vocabulary. A small consented local recording set could serve as a qualitative test.

Slide 53 | Six-week timeline (1:15)



WHAT TO SAY

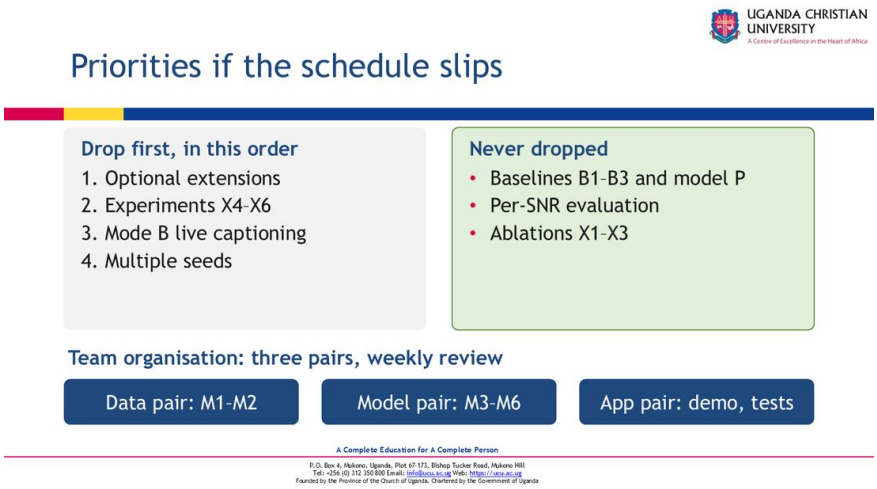
- The schedule covers six weeks and is ordered by dependency: no model before the data is verified, and no fusion experiment before both single-stream baselines work.
- Week 1: environment and dataset checks. Weeks 1 to 2: preprocessing. Mid weeks 2 to 3: the audio baseline. Weeks 3 to 4: encoders and the visual-only and concatenation baselines. Week 4: the fusion model. Weeks 4 to 5: training and tuning.
- The core system freezes at the end of week 5. Week 5 to 6: evaluation, ablations and the application. Final week: testing, documentation and the demo. Optional extensions only go into the slack after the freeze.
- Long training runs start as early as possible and overlap with developing the next component.

BACKGROUND YOU SHOULD UNDERSTAND

Each milestone has a completion criterion. Examples: M1, every member can run the test suite and GPU access is confirmed. M3, B1 overfits 8 clips and a full run is evaluated at all five SNRs. M5, no NaN under full-modality dropout. M8, Mode A matches the evaluation script and Mode B runs for 5 minutes without error. M9, a fresh clone reproduces one evaluation and the demo is rehearsed with a recorded fallback clip.

M8 (application) can start Mode A after M3, using the first checkpoint; it switches to the final model after M6.

Slide 54 | Priorities if the schedule slips (0:45)



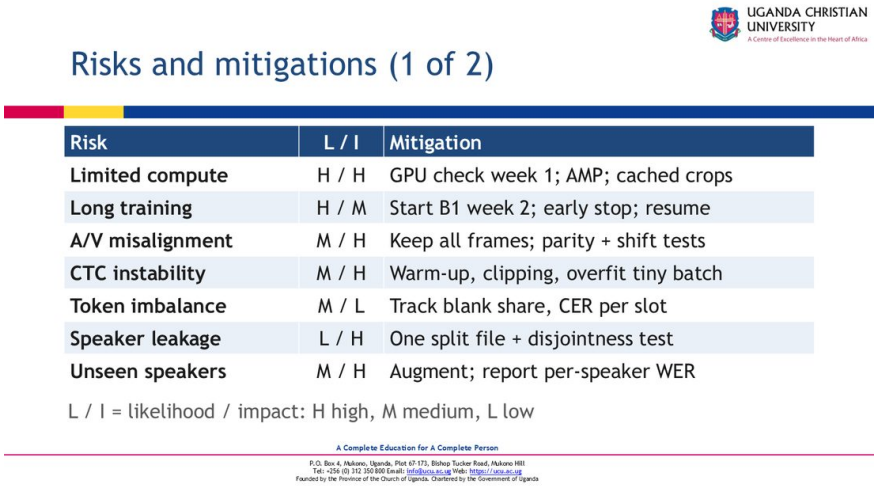
WHAT TO SAY

- If the schedule slips, we drop work in this order: first the optional extensions, then experiments X4 to X6, then Mode B live captioning, while Mode A stays, and then multiple seeds.
- Never dropped: the three baselines and the proposed model, the per-SNR evaluation, and ablations X1 to X3. Those are the core contribution in the proposal.
- We work in three pairs: a data pair on milestones 1 and 2, a model pair on encoders, fusion and training, and an application pair on inference, the Gradio interface, the live mode and tests. Everyone contributes to evaluation and the report, and a short weekly review checks completion criteria and re-plans.

BACKGROUND YOU SHOULD UNDERSTAND

Pair names are by role; the team will confirm which two members sit in each pair.

Slide 55 | Risks and mitigations (1 of 2) (1:00)



WHAT TO SAY

- We rated each risk by likelihood and impact.
- The highest is limited compute, high and high: we confirm GPU access in week 1, use mixed precision and cached crops, and develop on a subset. Long training: start the audio baseline in week 2, use early stopping and resumable checkpoints.
- Audio-video misalignment: never drop frames, plus the parity and time-shift tests. CTC instability: warm-up, clipping, and overfitting a tiny batch before full runs. Token imbalance:

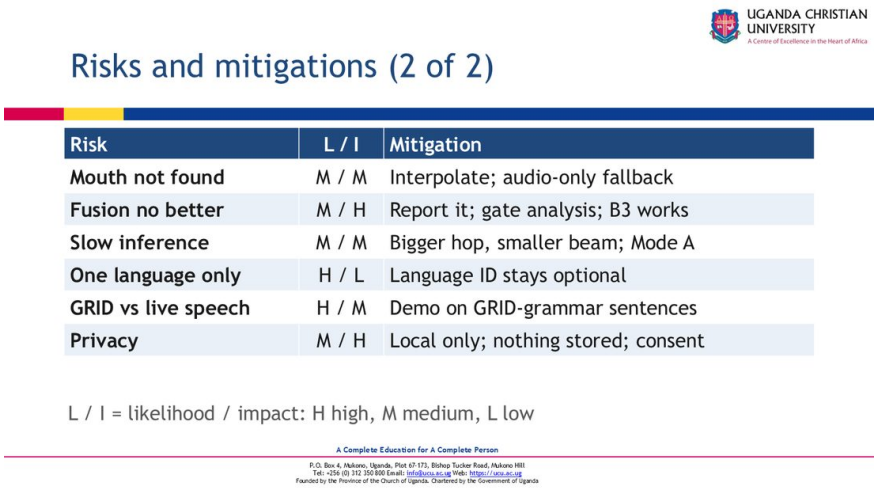
the blank dominates CTC outputs, so we track the blank share and CER per word slot. Speaker leakage: one split file and a disjointness test. Unseen speakers: augmentation and honest per-speaker results.

BACKGROUND YOU SHOULD UNDERSTAND

Full mitigations are in the risk table of the implementation document. Ratings are the team's judgement at planning stage and are reviewed weekly.

Token imbalance detail: GRID's grammar makes some characters much more frequent; the letter slot is inspected separately because it is the hardest.

Slide 56 | Risks and mitigations (2 of 2) (1:00)



WHAT TO SAY

- Mouth not found, from lighting, head pose or occlusion, mostly affects the live demo: we interpolate short gaps, fall back to audio-only, and set up good lighting and a frontal camera.
- Fusion may not beat audio-only: we report it truthfully and diagnose it, and B3 is still a working fused system. Slow inference: we increase the window hop, reduce the beam, and Mode A is the fallback.
- One language only: language ID stays optional. GRID versus live speech: we demo GRID-grammar sentences and name larger datasets as future work.
- Privacy: all input is processed locally, live video and audio are not stored by default, there is an on-screen indicator while capturing, any demo recording needs written consent and is deleted after the project, and nothing is uploaded to third-party services.

BACKGROUND YOU SHOULD UNDERSTAND

Synthetic additive noise does not reproduce reverberation, competing speakers or microphone distortion; this is a stated limitation.

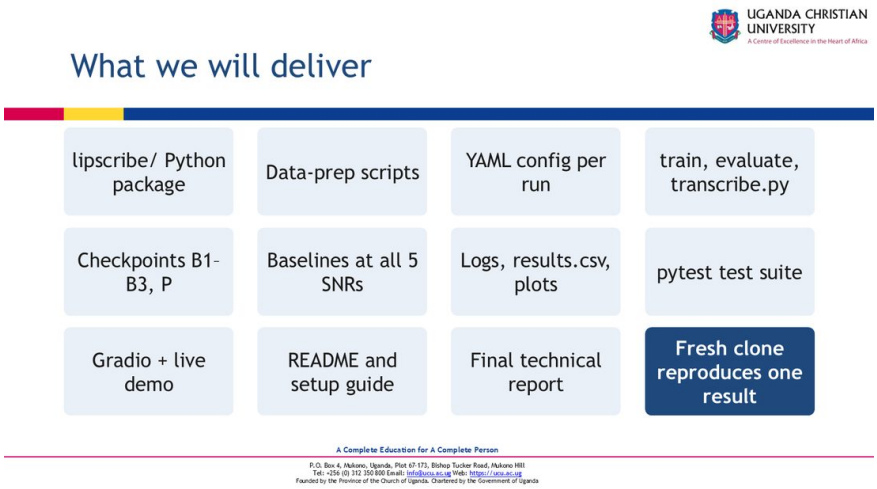
A frontal, unoccluded face is assumed by the visual front-end, an open problem in the literature [5], [4].

QUESTIONS YOU MAY BE ASKED

Why is privacy rated high impact?

Faces and voices are identifiable personal data. Even in a student demo, mishandling them would be a real harm, so we design for local processing and consent from the start.

Slide 57 | What we will deliver (0:45)



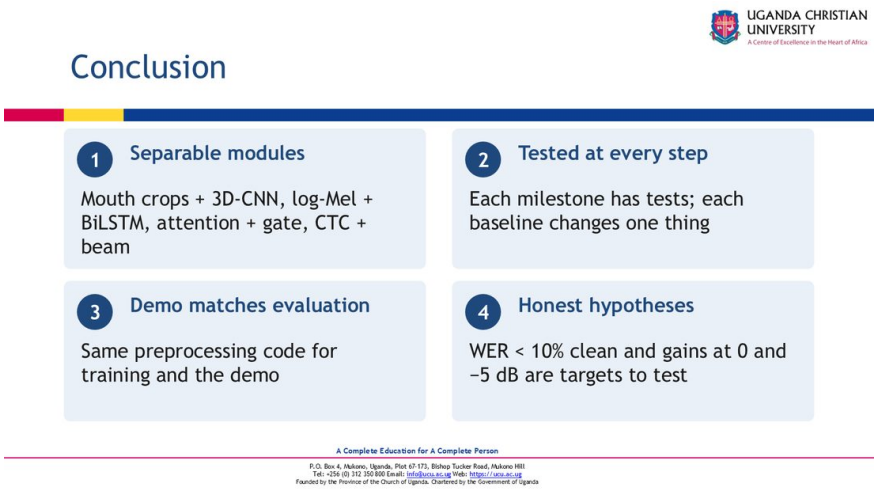
WHAT TO SAY

- Our deliverables: the lipscribe Python package with data, preprocessing, models, decoding, evaluation and app sub-packages; the data-preparation scripts; one YAML config per experiment; train, evaluate and transcribe scripts; checkpoints for the baselines and the proposed model; baselines evaluated at all five SNRs; logs, the results table and plots; the pytest suite; the Gradio and live demos; a README and setup guide; and the final technical report.
- The highlighted tile is the overall acceptance test: a team member not involved in setup reproduces one evaluation from a fresh clone.

BACKGROUND YOU SHOULD UNDERSTAND

Each deliverable has evidence that it works (deliverables table in the document). For example, re-running evaluate.py on a released checkpoint must reproduce its reported WER, and the live demo comes with a recorded fallback video.

Slide 58 | Conclusion (1:00)



WHAT TO SAY

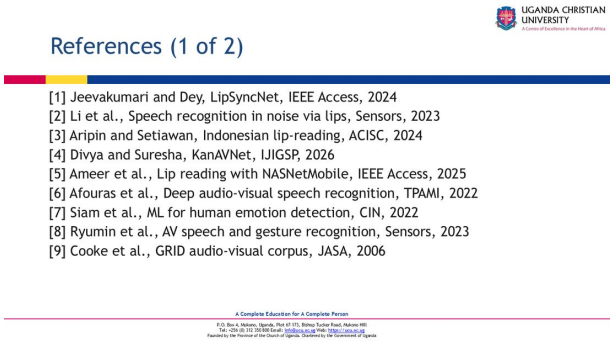
- To conclude. LipScribe is built from separable modules: mouth crops with a 3D-CNN, log-Mel with a BiLSTM, cross-modal attention with a gate, and CTC with beam search.
- Every milestone has its own tests, and every baseline changes exactly one thing, so each comparison isolates one effect.

- The same preprocessing code serves training and the demo, so what we demonstrate reflects what we evaluated.
- And we are honest about uncertainty: WER below 10% on clean audio and gains at 0 and -5 dB are hypotheses to test, not results. We will report outcomes faithfully, including any where fusion does not help.

BACKGROUND YOU SHOULD UNDERSTAND

The one-sentence version: captioning tools that only listen fail in noise; LipScribe also reads the lips and learns, through attention and modality dropout, to lean on them when the audio degrades; we will measure that on held-out speakers at five noise levels against three baselines.

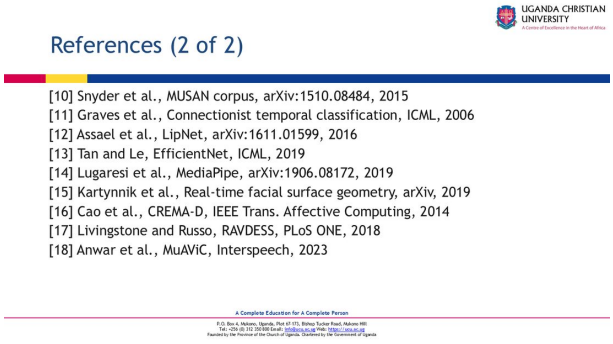
Slide 59 | References (1 of 2) (0:05)



WHAT TO SAY

- References 1 to 8 come from our proposal; 9 onwards were added for the implementation document.

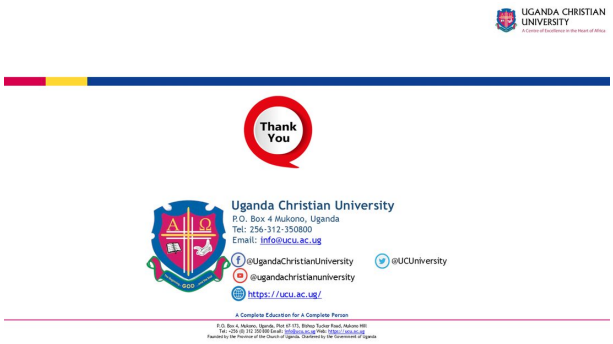
Slide 60 | References (2 of 2) (0:05)



WHAT TO SAY

- Full references with DOIs are in the implementation document.

Slide 61 | Thank you (–)



WHAT TO SAY

- Thank you. We are happy to take questions.

BACKGROUND YOU SHOULD UNDERSTAND

During questions, the presenter of the relevant part answers first; others add only if needed. Any member may be asked about any part.